

Capitolo 02 - Programmazione orientata agli oggetti in Java

◆ Classi in Java

La **classe** è uno dei concetti fondamentali della **programmazione orientata agli oggetti (OOP)**. Una classe è un **modello (blueprint)** per la creazione di **oggetti**, che rappresentano entità con **stato** (attributi) e **comportamenti** (metodi).

◆ Caratteristiche principali

- Una **classe** non consuma memoria da sola: occupa memoria solo quando vengono create istanze (`new`).
- Una classe può essere **istanziata più volte**, producendo oggetti distinti.
- Ogni classe dovrebbe rispettare il **principio di responsabilità singola (SRP)**: fare bene una sola cosa.

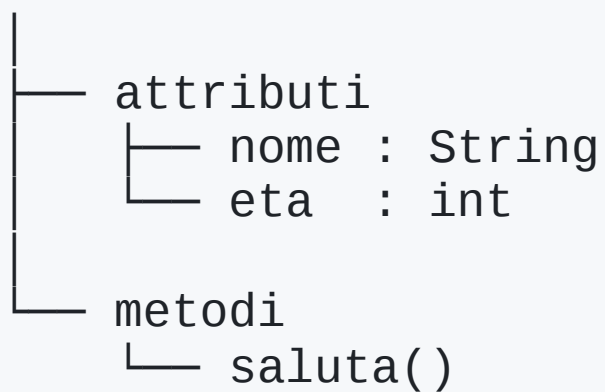
◆ Java è orientato agli oggetti

- **Classe**: definisce un nuovo tipo, con **proprietà** (attributi) e **azioni** (metodi).
- **Oggetto**: istanza della classe, con dati propri e metodi applicabili a essi.
- In Java quasi tutto è un oggetto.
 - Fanno eccezione i tipi primitivi
 - (int, double, boolean, char, ...).
 - Gli array sono oggetti a tutti gli effetti,
 - anche se il linguaggio li gestisce con una sintassi speciale.

◆ Le classi definiscono

- **Attributi** (o campi) → dati che descrivono lo stato dell'oggetto.
- **Metodi** → operazioni che l'oggetto può eseguire o che agiscono sui dati.

Classe Persona



Esempi:

- Definite dal programmatore (`Automobile` , `Studente` , `Persona` ...).
- Definite da Java (`String` , `System` , `Scanner` ...).

◆ Creazione di oggetti

La creazione di un oggetto si chiama **istanza** e avviene con `new`.

```
Persona p1 = new Persona();  
p1.nome = "Luca";  
p1.cognome = "Rossi";  
p1.eta = 25;
```

Nota

La classe rappresenta il progetto dell'oggetto.

L'oggetto viene creato in memoria quando viene utilizzato l'operatore new.

Una stessa classe può generare molti oggetti diversi e indipendenti tra loro.

◆ Struttura di una classe

```
package model;

public class Persona {

    String nome;
    String cognome;
    int eta;

    // metodo di istanza
    public void saluta() {
        System.out.println("Ciao, sono " + nome);
    }
}
```

◆ Regole di base per le classi

- Ogni classe deve stare in un file con **lo stesso nome** e estensione `.java`.
- Una classe può contenere:
 - **Attributi**: variabili che rappresentano lo stato.
 - **Metodi**: azioni sugli oggetti.
- Per convenzione:
 - i **nomi delle classi** iniziano con la maiuscola (`Persona`),
 - i **metodi** con la minuscola (`saluta`).

◆ Documentazione e libreria standard

- Java fornisce una ricca **libreria di classi pronte all'uso** (principio DRY: *Don't Repeat Yourself*).
- Ogni classe della libreria è documentata con API ufficiali.
- Dalla **Java 9** la libreria standard è stata organizzata in **moduli**.

◆ Tipologie di classi in Java

In Java, possiamo distinguere le classi in due grandi categorie:

1. **Classi istanziabili** → possono generare oggetti con `new`.
2. **Classi non istanziabili** → non hanno senso come oggetti o non possono essere create con `new`.

👉 Questa distinzione vale sia per le classi della libreria standard, sia per le classi definite da noi.

◆ Classi istanziabili

- Forniscono il **prototipo** di una famiglia di oggetti che condividono struttura e comportamento.
- Ogni oggetto (istanza) ha **identità propria** e contiene i propri valori per gli attributi.
- I metodi operano sui dati dell'istanza.
- Tipicamente hanno **costruttori** e attributi/metodi **non statici**.

Esempio

```
class Persona {  
    String nome;  
    int eta;  
  
    void saluta() {  
        System.out.println("Ciao, mi chiamo " + nome);  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Persona p1 = new Persona("Luca", 25);  
        p1.saluta(); // "Ciao, mi chiamo Luca"  
    }  
}
```

◆ Classi non istanziabili

- Raccolgono **metodi e costanti statici** utili in generale, senza legame con istanze.
- Non hanno senso come oggetti singoli (es. non esistono “più matematiche”).
- Sono usate per **utility**, **costanti** o per definire un **punto d'ingresso** (`main`).

Esempi

- `Math` → metodi matematici (`Math.sqrt(2)`) e costanti (`Math.PI`).
- `System` → accesso al sistema (`System.out.println(...)`).

```
System.out.println(Math.sqrt(9)); // 3.0  
System.out.println(Math.PI);     // 3.1415...
```


◆ Classi ibride

Alcune classi **istanziabili** includono anche metodi statici.

-

Esempio: `String` → è istanziabile (`new String("abc")`) o letterale `"abc"`), ma ha anche metodi statici (`String.valueOf(...)`).

👉 Questo **ibridismo** va usato con cautela:

- È accettabile avere **costanti statiche** o pochi metodi statici di utilità.
- Ma mischiare troppi elementi statici e dinamici può rendere il design poco chiaro.

 Documentazione ufficiale:

- [Java 8](#)
- [Java 11](#)
- [Java 17](#)
- [Java 21](#)

✓ Riepilogo

- Una **classe** è un progetto (blueprint) → gli **oggetti** sono istanze reali.
- Ogni oggetto ha **proprietà** (attributi) e **comportamenti** (metodi).
- Le classi possono essere create dal programmatore o già pronte nella libreria Java.
- Buona pratica: seguire il **principio di responsabilità singola (SRP)**.
- **Classi istanziabili** → rappresentano concetti con molteplici versioni individuali (`Persona` , `String` , `ArrayList`).
- **Classi non istanziabili** → rappresentano concetti unici, usate per metodi e costanti (`Math` , `System`).
- **Classi ibride** → istanziabili con qualche elemento statico (`String`).

Domande

Che cos'è una classe in Java?

- A. Una variabile che contiene più valori
- B. Un insieme di istruzioni che definisce un tipo di oggetto
- C. Un metodo speciale
- D. Una struttura per gestire array

Che cos'è un oggetto in Java?

- A. Una variabile di tipo primitivo**
- B. Un'istanza di una classe**
- C. Un metodo statico**
- D. Una costante**

Quale parola chiave viene usata per creare un nuovo oggetto in Java?

- A. class
- B. this
- C. new
- D. void

Cosa rappresentano gli attributi di una classe?

- A. Le azioni che un oggetto può compiere**
- B. Lo stato o le proprietà dell'oggetto**
- C. Il costruttore della classe**
- D. Le regole di visibilità**

Quale elemento di una classe rappresenta lo stato dell'oggetto?

- A. Metodo
- B. Attributo
- C. Package
- D. Import

Quale convenzione si usa per il nome delle classi in Java?

A. Tutte maiuscole

B. Tutte minuscole

C. PascalCase (iniziale maiuscola)

D. snake_case

Quanti oggetti posso creare a partire da una classe?

A. Uno solo

B. Nessuno

C. Un numero qualsiasi

D. Solo se la classe è statica

Quali tra i seguenti NON sono oggetti in Java?

A. ArrayList

B. String

C. int

D. Scanner

Quale principio di progettazione dice che una classe deve fare una sola cosa?

A. DRY

B. SRP (Single Responsibility Principle)

C. OCP

D. LSP

Esercizi

Esercizio 1

Disegna la struttura di una classe `Libro` indicando attributi e metodi.

Esercizio 2

Definisci una classe `Automobile` contenente:

- `marca`
- `modello`
- `cilindrata`

e un metodo:

```
void descrivi()
```

Esercizio 3

Classifica le seguenti classi:

- String
- Math
- System
- ArrayList

come:

- istanziabile
- non istanziabile
- ibrida

Esercizio 4

Progetta una classe **Studente**
indicando almeno:

- 4 attributi
- 3 metodi

senza scrivere codice Java.

0---

◆ Oggetti e Istanze in Java

Una **classe** definisce un modello generale, mentre un **oggetto** rappresenta una realizzazione concreta di quel modello.

Quando creiamo un oggetto a partire da una classe, diciamo che stiamo creando una **istanza** della classe.

◆ Classe e oggetto

Possiamo immaginare una classe come il progetto di una casa.

Classe → Progetto

Oggetto → Casa costruita

Oppure:

Classe Persona

```
graph LR; C[Classe Persona] --- O1[Oggetto p1]; C --- O2[Oggetto p2]; C --- O3[Oggetto p3];
```

Oggetto p1
Oggetto p2
Oggetto p3

Una sola classe può generare molti oggetti diversi.

◆ Creazione di un oggetto

Gli oggetti vengono creati tramite l'operatore `new` .

```
Persona p1 = new Persona();
```

In questo

esempio:

- `Persona` è il tipo della variabile.
- `p1` è il riferimento all'oggetto.
- `new Persona()` crea una nuova istanza.

◆ Oggetti indipendenti

Ogni oggetto possiede il proprio stato.

```
Persona p1 = new Persona();  
Persona p2 = new Persona();  
  
p1.nome = "Luca";  
p2.nome = "Anna";  
  
System.out.println(p1.nome);  
System.out.println(p2.nome);
```

Output:

```
Luca  
Anna
```

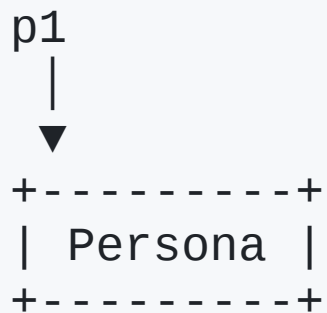
Le modifiche a un oggetto non influenzano gli altri.

◆ Riferimento e oggetto

Una variabile oggetto non contiene direttamente l'oggetto.

Contiene invece un **riferimento** all'oggetto creato in memoria.

```
Persona p1 = new Persona();
```

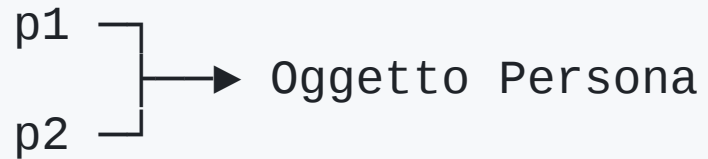


Per questo motivo diciamo che `p1` è una **variabile riferimento**.

◆ Più riferimenti allo stesso oggetto

Due variabili possono riferirsi allo stesso oggetto.

```
Persona p1 = new Persona();  
Persona p2 = p1;
```



Se modifico l'oggetto tramite `p1` , vedrò le modifiche anche tramite `p2` .

```
p1.nome = "Luca";  
System.out.println(p2.nome);
```

Output:

```
Luca
```

◆ Confronto tra riferimenti

L'operatore `==` confronta i riferimenti.

```
Persona p1 = new Persona();  
Persona p2 = new Persona();  
  
System.out.println(p1 == p2);
```

Output:

```
false
```


Gli oggetti sono diversi.

◆ Valore null

Una variabile riferimento può non puntare ad alcun oggetto.

```
Persona p1 = null;
```

In questo caso:



A diagram illustrating the state of a variable. The text 'p1' is at the top, followed by a vertical line with a downward-pointing arrow, and the text 'null' at the bottom.

Tentare di usare un riferimento nullo genera un errore.

```
p1.saluta();
```

```
NullPointerException
```

◆ Ciclo di vita degli oggetti

Un oggetto:

1. viene creato con `new` ;
2. viene utilizzato dal programma;
3. quando non è più raggiungibile può essere eliminato automaticamente dal Garbage Collector.

Creazione



Utilizzo



Garbage Collection

◆ Buone pratiche

- Utilizzare nomi significativi per gli oggetti.
- Evitare riferimenti inutilizzati.
- Inizializzare gli oggetti prima dell'uso.
- Prestare attenzione ai valori `null`.

Riepilogo

- Un oggetto è un'istanza di una classe.
- Gli oggetti vengono creati tramite `new`.
- Una variabile oggetto contiene un riferimento.
- Più riferimenti possono puntare allo stesso oggetto.
- `null` indica l'assenza di un oggetto.
- Gli oggetti non più utilizzati vengono rimossi dal Garbage Collector.

Domande

Che cos'è un'istanza di una classe?

- A. Un metodo
- B. Un attributo
- C. Un oggetto creato dalla classe
- D. Un package

Quale operatore viene utilizzato per creare un oggetto?

- A. class
- B. new
- C. this
- D. instanceof

Cosa contiene una variabile oggetto?

- A. L'intero contenuto dell'oggetto
- B. Un riferimento all'oggetto
- C. Un metodo
- D. Una copia della classe

Quale valore indica l'assenza di un oggetto?

- A. void
- B. empty
- C. false
- D. null

Cosa accade se due variabili fanno riferimento allo stesso oggetto?

- A. Vengono creati due oggetti
- B. Gli oggetti vengono copiati
- C. Entrambe accedono allo stesso oggetto
- D. Si genera un errore

Esercizi

Esercizio 1

Disegna il diagramma dei riferimenti per:

```
Persona p1 = new Persona();
```

Esercizio 2

Disegna il diagramma dei riferimenti per:

```
Persona p1 = new Persona();  
Persona p2 = p1;
```

Esercizio 3

Scrivi una classe `Libro` e crea due oggetti distinti.

Esercizio 4

Crea una classe `Studente` e tre istanze differenti.

Esercizio 5

Spiega con parole tue la differenza tra:

```
Persona p1 = new Persona();  
Persona p2 = new Persona();
```

e

```
Persona p1 = new Persona();  
Persona p2 = p1;
```

I metodi Costruttori

◆ Introduzione

Dopo aver visto come creare un oggetto tramite `new`, vediamo come inizializzarlo correttamente.

Il costruttore è un metodo speciale che viene eseguito automaticamente quando viene creato un oggetto.

◆ Cos'è un costruttore

Caratteristiche:

- stesso nome della classe
- nessun tipo di ritorno
- eseguito automaticamente
- inizializza lo stato dell'oggetto

```
class Persona {  
    String nome;  
    int eta;  
  
    Persona() {  
        System.out.println("Nuova persona creata");  
    }  
}
```

◆ Costruttore di default

Se non definiamo alcun costruttore:

```
class Persona {  
    String nome;  
    int eta;  
}
```

Java genera automaticamente:

```
Persona() {  
}
```

◆ Valori di default degli attributi

Tipo	Valore
int	0
double	0.0
boolean	false
char	'\u0000'
riferimento	null

Esempio:

```
Persona p = new Persona();  
---  
  
System.out.println(p.nome); // null  
System.out.println(p.eta);  // 0
```


◆ Costruttore personalizzato

```
class Persona {  
    String nome;  
    int eta;  
  
    Persona(String nome, int eta) {  
        this.nome = nome;  
        this.eta = eta;  
    }  
}
```

Utilizzo:

```
Persona p = new Persona("Luca", 25);
```

◆ La parola chiave this

Rappresenta l'oggetto corrente.

```
Persona(String nome, int eta) {  
    this.nome = nome;  
    this.eta = eta;  
}
```

Schema:

```
Parametro nome  
  |  
  ▼  
this.nome
```

◆ Overloading dei costruttori

Una classe può avere più costruttori.

```
class Persona {  
  
    String nome;  
    int eta;  
  
    Persona() {  
    }  
  
    Persona(String nome) {  
        this.nome = nome;  
    }  
    ---  
  
    Persona(String nome, int eta) {  
        this.nome = nome;  
        this.eta = eta;  
    }  
}
```

◆ Operatore new

Quando scriviamo:

```
Persona p = new Persona("Luca", 25);
```

Java:

1. alloca memoria;
2. crea l'oggetto;
3. esegue il costruttore;
4. restituisce il riferimento.

```
new
├── crea oggetto
├── esegue costruttore
└── restituisce riferimento
```

◆ Buone pratiche

- inizializzare sempre gli attributi importanti;
- usare nomi significativi per i parametri;
- evitare costruttori con troppi parametri;
- usare `this` quando migliora la leggibilità.

✓ Riepilogo

- Il costruttore inizializza un oggetto.
- Ha lo stesso nome della classe.
- Non possiede tipo di ritorno.
- Viene chiamato automaticamente da `new`.
- È possibile definire più costruttori (overloading).
- `this` rappresenta l'oggetto corrente.

Domande

```
Che cos'è un costruttore in Java?
A. Un metodo che distrugge gli oggetti
B. Un metodo speciale che inizializza gli oggetti
C. Un attributo statico
D. Un package di sistema

---

Quale caratteristica distingue un costruttore da un normale metodo?
A. Deve essere statico
B. Deve restituire un valore
C. Ha lo stesso nome della classe
D. È sempre privato

---

Quale tipo di ritorno possiede un costruttore?
A. void
B. int
C. Object
D. Nessuno

---

Quando viene eseguito un costruttore?
A. Alla compilazione
B. Quando viene creato un oggetto
C. Alla chiusura del programma
D. Alla chiamata di un metodo statico

---

Quale parola chiave provoca la chiamata di un costruttore?
A. class
B. this
C. new
D. return

---

Cosa accade se una classe non definisce alcun costruttore?
A. Il programma non compila
B. Java genera automaticamente un costruttore di default
C. Tutti gli attributi diventano obbligatori
D. La classe diventa astratta

---

Quale valore di default viene assegnato a un attributo di tipo int?
A. null
B. false
C. 0
D. 1

---

A cosa serve la parola chiave this all'interno di un costruttore?
A. A creare un nuovo oggetto
B. A richiamare il Garbage Collector
C. A riferirsi all'oggetto corrente
D. A dichiarare un attributo statico

---

Come si chiama la possibilità di avere più costruttori nella stessa classe?
A. Override
B. Encapsulation
C. Overloading
D. Inheritance

---

Quale delle seguenti dichiarazioni rappresenta correttamente un costruttore della classe Persona?
A. public void Persona() {}
B. public int Persona() {}
C. public Persona() {}
D. public static Persona() {}

---
```


Esercizi

Esercizio 1

Creare una classe `Libro` con un costruttore vuoto.

Esercizio 2

Creare una classe `Studente` con un costruttore che riceva:

- nome
- cognome
- matricola

Esercizio 3

Creare tre costruttori diversi per una classe `Automobile` .

Esercizio 4

Spiegare il ruolo di `this` .

Esercizio 5

Disegnare il flusso delle operazioni eseguite da:

```
Persona p = new Persona("Luca", 25);
```

Questa sequenza è molto naturale perché segue il percorso:

```
110 Classi  
↓  
120 Oggetti e Istanze  
↓  
130 Costruttori  
↓  
140 Getter e Setter  
↓  
150 Incapsulamento
```

◆ **Getter e Setter in Java**

Dopo aver visto come creare e inizializzare gli oggetti, è importante imparare a gestire correttamente l'accesso ai loro dati.

I **getter** e i **setter** sono metodi speciali che permettono di leggere e modificare gli attributi di una classe in modo controllato.

Sono uno degli strumenti fondamentali della programmazione orientata agli oggetti.

◆ Perché usare Getter e Setter

Supponiamo di avere una classe:

```
class Persona {  
    String nome;  
    int eta;  
}
```

Gli attributi possono essere modificati direttamente:

```
Persona p = new Persona();  
---  
  
p.nome = "Luca";  
p.eta = 25;
```

Questo approccio è semplice, ma non permette di controllare i valori assegnati.

◆ Il Getter

Un **getter** è un metodo che restituisce il valore di un attributo.

```
class Persona {  
    private String nome;  
  
    public String getNome() {  
        return nome;  
    }  
}
```

Utilizzo:

```
Persona p = new Persona();  
  
System.out.println(p.getNome());
```

Schema:

◆ Il Setter

Un **setter** è un metodo che modifica il valore di un attributo.

```
class Persona {  
    private String nome;  
  
    public void setName(String nome) {  
        this.nome = nome;  
    }  
}
```

Utilizzo:

```
Persona p = new Persona();  
  
p.setName("Luca");
```

Schema:

◆ Esempio completo

```
class Persona {  
    private String nome;  
  
    public String getNome() {  
        return nome;  
    }  
  
    public void setNome(String nome) {  
        this.nome = nome;  
    }  
}
```

Utilizzo:

```
Persona p = new Persona();  
  
p.setNome("Anna");  
  
System.out.println(p.getNome());
```

Output:

```
Anna
```

◆ Controllo dei dati nei Setter

Uno dei principali vantaggi dei setter è la possibilità di validare i dati.

```
class ContoBancario {  
    private double saldo;  
  
    public double getSaldo() {  
        return saldo;  
    }  
  
    public void setSaldo(double saldo) {  
        if (saldo >= 0) {  
            this.saldo = saldo;  
        }  
    }  
}
```

Utilizzo:

```
ContoBancario conto = new ContoBancario();  
  
conto.setSaldo(1000);  
conto.setSaldo(-500); // ignorato
```

◆ Convenzioni di Naming

In Java si utilizzano convenzioni standard:

Attributo	Getter	Setter
nome	getNome()	setNome()
cognome	getCognome()	setCognome()
saldo	getSaldo()	setSaldo()

Per gli attributi boolean:

```
private boolean attivo;
```

si usa spesso:

```
isAttivo()
```

anziché:

```
getAttivo()
```


◆ **Getter e Setter non sono obbligatori**

Non tutti gli attributi devono avere sia getter che setter.

Esempio:

```
private String codiceFiscale;
```

Potrei permettere la lettura:

```
public String getCodiceFiscale()
```

ma impedire la modifica eliminando il setter.

◆ Anticipazione: Incapsulamento

Nella maggior parte dei casi gli attributi vengono dichiarati:

```
private
```

e vengono resi accessibili tramite getter e setter.

Questo principio prende il nome di **incapsulamento** e verrà approfondito nel prossimo capitolo.

Riepilogo

- I getter leggono il valore di un attributo.
- I setter modificano il valore di un attributo.
- Permettono di controllare l'accesso ai dati.
- Consentono di validare i valori inseriti.
- Seguono convenzioni di naming standard.
- Sono uno degli strumenti principali dell'incapsulamento.

Domande

Che cosa fa un getter?

- A. Modifica un attributo
- B. Distrugge un oggetto
- C. Restituisce il valore di un attributo
- D. Crea una nuova istanza

Che cosa fa un setter?

- A. Restituisce il valore di un attributo
- B. Modifica il valore di un attributo
- C. Crea un oggetto
- D. Elimina un attributo

Quale convenzione si usa per leggere l'attributo nome?

- A. nome()
- B. readNome()
- C. getNome()
- D. nomeGet()

Quale convenzione si usa normalmente per un attributo boolean attivo?

- A. getAttivo()
- B. attivo()
- C. setAttivo()
- D. isAttivo()

Perché si utilizzano i setter?

- A. Per compilare più velocemente
- B. Per controllare i valori assegnati
- C. Per evitare i costruttori
- D. Per sostituire le classi

Quale modificatore viene normalmente utilizzato per gli attributi gestiti tramite getter e setter?

- A. static
- B. final
- C. private
- D. protected

Un setter può contenere controlli sui dati?

- A. No
- B. Solo nelle classi astratte
- C. Solo nei record
- D. Sì

Quale valore restituisce un getter?

- A. Sempre void
- B. Il valore dell'attributo associato
- C. Un nuovo oggetto
- D. Un riferimento casuale

Getter e setter sono:

- A. Tipi primitivi
- B. Costruttori
- C. Metodi
- D. Package

Getter e setter aiutano a implementare:

- A. Overloading
- B. Casting
- C. Incapsulamento
- D. Polimorfismo

Esercizi

Esercizio 1

Creare una classe `Libro` con:

- titolo
- autore

e relativi getter e setter.

Esercizio 2

Creare una classe `Studente` con:

- nome
- cognome
- matricola

e relativi getter e setter.

Esercizio 3

Creare una classe `ContoCorrente` che impedisca l'inserimento di un saldo negativo.

Esercizio 4

Progettare una classe contenente un attributo leggibile ma non modificabile.

Esercizio 5

Spiegare con parole proprie perché i getter e setter sono preferibili agli attributi pubblici.

◆ Incapsulamento in Java

L'**incapsulamento** è uno dei principi fondamentali della programmazione orientata agli oggetti.

Consiste nel **nascondere i dettagli interni di una classe** e nel permettere l'accesso ai dati soltanto attraverso un'interfaccia controllata.

L'obiettivo è proteggere lo stato degli oggetti e ridurre le dipendenze tra le varie parti del programma.

◆ Interfaccia e Implementazione

Ogni classe può essere vista come composta da due parti:

```
+-----+  
|      Interfaccia      |  
|-----|  
|  metodi pubblici  |  
+-----+
```

```
+-----+  
|      Implementazione   |  
|-----|  
|  attributi           |  
|  logica interna      |  
+-----+
```

L'utente della classe dovrebbe conoscere soltanto l'interfaccia pubblica.
I dettagli interni possono cambiare senza influenzare il codice esterno.

◆ Perché usare l'incapsulamento

L'incapsulamento offre numerosi vantaggi:

- maggiore sicurezza dei dati;
- riduzione degli errori;
- codice più robusto;
- minore accoppiamento tra le classi;
- maggiore facilità di manutenzione.

◆ Esempio senza incapsulamento

```
class ContoCorrente {  
    public double saldo;  
}
```

Utilizzo:

```
ContoCorrente conto = new ContoCorrente();  
  
conto.saldo = -1000;
```

In questo caso qualsiasi valore può essere assegnato al saldo.

◆ Esempio con incapsulamento

```
class ContoCorrente {  
    private double saldo;  
  
    public double getSaldo() {  
        return saldo;  
    }  
  
    public void deposita(double importo) {  
        if (importo > 0) {  
            saldo += importo;  
        }  
    }  
}
```

Utilizzo:

```
ContoCorrente conto = new ContoCorrente();  
conto.deposita(1000);
```

L'oggetto mantiene il controllo sul proprio stato.

◆ I modificatori di accesso

Java mette a disposizione quattro livelli di visibilità.

Modificatore	Stessa classe	Stesso package	Sottoclasse	Ovunque
private	✓	✗	✗	✗
default	✓	✓	✗	✗
protected	✓	✓	✓	✗
public	✓	✓	✓	✓

◆ Private

Un elemento dichiarato `private` è accessibile soltanto all'interno della stessa classe.

```
class Persona {  
    private String nome;  
}
```

Questo è il modificatore più utilizzato per gli attributi.

◆ Public

Un elemento dichiarato `public` è accessibile da qualsiasi altra classe.

```
public void saluta() {  
    System.out.println("Ciao");  
}
```

I metodi che costituiscono l'interfaccia della classe sono spesso pubblici.

◆ Protected

Il modificatore `protected` verrà approfondito nel capitolo dedicato all'ereditarietà.

Permette l'accesso:

- alla classe stessa;
- alle classi dello stesso package;
- alle sottoclassi.

```
protected String descrizione;
```


◆ Package-Private

Se non viene specificato alcun modificatore:

```
class Studente {  
}
```

oppure

```
String nome;
```

l'elemento è visibile soltanto all'interno dello stesso package.

◆ Modificare l'implementazione senza modificare l'interfaccia

Supponiamo di avere:

```
class Rettangolo {  
  
    private int base;  
    private int altezza;  
  
    public int getArea() {  
        return base * altezza;  
    }  
}
```

Il codice esterno utilizza:

```
rettangolo.getArea();
```

Se in futuro cambiamo il modo in cui viene calcolata l'area, il codice che utilizza la classe non dovrà essere modificato.

Questo è uno dei principali vantaggi dell'incapsulamento.

◆ Buone pratiche

- dichiarare gli attributi come `private` ;
- esporre solo i metodi realmente necessari;
- evitare attributi pubblici;
- validare i dati prima di modificarli;
- mantenere separati interfaccia e implementazione.

Riepilogo

- L'incapsulamento protegge lo stato degli oggetti.
- Consente di separare interfaccia e implementazione.
- Gli attributi dovrebbero essere generalmente `private`.
- L'accesso ai dati avviene tramite metodi pubblici.
- I modificatori di accesso controllano la visibilità degli elementi.
- L'incapsulamento rende il codice più robusto e manutenibile.

Domande

Che cos'è l'incapsulamento?

- A. La creazione di più oggetti
- B. La possibilità di usare più costruttori
- C. Il nascondimento dei dettagli interni di una classe
- D. La conversione dei tipi

Quale modificatore rende un attributo accessibile solo nella stessa classe?

- A. public
- B. protected
- C. default
- D. private

Quale modificatore permette l'accesso da qualsiasi classe?

- A. private
- B. public
- C. protected
- D. default

Qual è il modificatore normalmente utilizzato per gli attributi?

- A. public
- B. protected
- C. private
- D. static

Quale vantaggio offre l'incapsulamento?

- A. Riduce la dimensione dei file
- B. Aumenta la velocità di compilazione
- C. Protegge i dati e riduce l'accoppiamento
- D. Elimina la necessità dei metodi

Che cosa rappresenta l'interfaccia di una classe?

- A. Gli attributi privati
- B. La memoria occupata
- C. Gli elementi utilizzabili dall'esterno
- D. Il package della classe

Cosa significa package-private?

- A. Visibile ovunque
- B. Visibile solo nel package corrente
- C. Visibile solo nelle sottoclassi
- D. Visibile solo nella classe

Quale modificatore verrà utilizzato spesso con l'ereditarietà?

- A. private
- B. final
- C. static
- D. protected

Perché evitare attributi pubblici?

- A. Occupano più memoria
- B. Violano l'incapsulamento
- C. Rendono il codice più lento
- D. Impediscono l'uso dei costruttori

L'incapsulamento permette di:

- A. Modificare l'implementazione senza cambiare l'interfaccia
- B. Eliminare gli oggetti
- C. Evitare i package
- D. Sostituire le classi astratte

Esercizi

Esercizio 1

Creare una classe `Libro` con tutti gli attributi privati.

Esercizio 2

Creare una classe `ContoCorrente` che permetta depositi ma non saldi negativi.

Esercizio 3

Indicare il livello di visibilità corretto per:

- attributi;
- getter;
- setter;
- metodi di utilità interni.

Esercizio 4

Trasformare una classe con attributi pubblici in una classe incapsulata.

Esercizio 5

Spiegare con parole proprie la differenza tra:

```
public double saldo;
```

e

```
private double saldo;
```

◆ Astrazione in Java

L'**astrazione** è uno dei principi fondamentali della programmazione orientata agli oggetti.

Consiste nel rappresentare un oggetto mettendo in evidenza le caratteristiche importanti e ignorando i dettagli non rilevanti.

In altre parole:

Ci concentriamo su COSA fa un oggetto,
non su COME lo fa.

L'astrazione permette di ridurre la complessità e di modellare il mondo reale in modo più semplice.

◆ Astrazione nella vita quotidiana

Utilizziamo continuamente oggetti astratti.

Ad esempio, quando guidiamo un'automobile:

```
Volante  
Pedali  
Cambio
```

sappiamo come usarli, ma non è necessario conoscere nel dettaglio il funzionamento del motore.

L'automobile ci offre un'interfaccia semplice nascondendo la complessità interna.

◆ Astrazione nella programmazione

Quando progettiamo una classe dobbiamo scegliere quali caratteristiche siano importanti.

Consideriamo una classe `Studente`.

```
Studente
|
├── nome
├── cognome
├── matricola
└── mediaVoti()
```


Non ci interessa rappresentare:

```
altezza della sedia  
colore dello zaino  
marca della penna
```

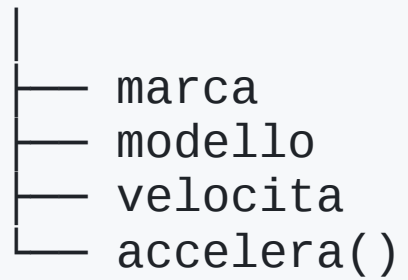
perché non sono rilevanti nel dominio applicativo.

◆ Modellare il dominio

L'astrazione consiste nel selezionare le informazioni significative.

Esempio:

Automobile



```
graph LR; A[Automobile] --- B[marca]; A --- C[modello]; A --- D[velocita]; A --- E[accelera()];
```

marca
modello
velocita
accelera()

Non è necessario rappresentare ogni singolo componente meccanico.

◆ Livelli di astrazione

Uno stesso oggetto può essere descritto a livelli diversi.

Livello alto

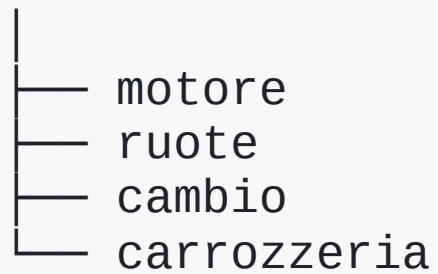
Veicolo

Livello intermedio

Automobile
Moto
Camion

Livello dettagliato

Automobile



La scelta del livello corretto dipende dal problema che vogliamo risolvere.

◆ **Astrazione e Incapsulamento**

Spesso vengono confusi, ma sono concetti diversi.

Astrazione

Risponde alla domanda:

Cosa rappresento?

e permette di concentrarsi sugli aspetti importanti.

Incapsulamento

Risponde alla domanda:

Come proteggo i dati?

e controlla l'accesso agli attributi e ai metodi.

◆ Esempio

Classe poco astratta:

```
class Studente {  
    String nome;  
    String cognome;  
    String colorePenna;  
    String marcaZaino;  
    int numeroScarpe;  
}
```

Classe meglio astratta:

```
class Studente {  
    String nome;  
    String cognome;  
    String matricola;  
}
```

La seconda rappresenta meglio il dominio applicativo.

◆ Astrazione in Java

Java offre diversi strumenti per implementare l'astrazione.

Tra questi:

- classi;
- metodi;
- package;
- classi astratte;
- interfacce.

Le classi astratte e le interfacce verranno approfondite nei capitoli successivi.

◆ Buone pratiche

- modellare solo ciò che è rilevante;
- evitare attributi inutili;
- scegliere nomi significativi;
- mantenere le classi semplici;
- concentrarsi sul problema da risolvere.

Riepilogo

- L'astrazione riduce la complessità.
- Permette di rappresentare solo le caratteristiche importanti.
- Aiuta a modellare correttamente il dominio applicativo.
- Si concentra sul "cosa" piuttosto che sul "come".
- È diversa dall'incapsulamento.
- Java offre diversi strumenti per realizzarla.

Domande

Che cos'è l'astrazione?

- A. La protezione degli attributi
- B. La creazione di oggetti
- C. La rappresentazione degli aspetti rilevanti di un oggetto
- D. L'ereditarietà tra classi

L'astrazione permette di:

- A. Aumentare il numero di attributi
- B. Ridurre la complessità
- C. Eliminare i metodi
- D. Evitare l'uso delle classi

Su cosa si concentra principalmente l'astrazione?

- A. Sul come funziona internamente un oggetto
- B. Sul dove viene memorizzato un oggetto
- C. Sul cosa rappresenta un oggetto
- D. Sul compilatore Java

Quale delle seguenti informazioni è più adatta a una classe `Studente`?

- A. Colore della penna
- B. Marca dello zaino
- C. Matricola
- D. Numero di scarpe

Quale principio OOP aiuta a modellare il dominio?

- A. Astrazione
- B. Casting
- C. Overloading
- D. Garbage Collection

Astrazione e incapsulamento sono:

- A. La stessa cosa
- B. Due concetti differenti
- C. Due tipi di ereditarietà
- D. Due tipi di costruttori

Quale domanda si pone l'astrazione?

- A. Come proteggere i dati?
- B. Come allocare memoria?
- C. Cosa rappresentare?
- D. Come compilare il codice?

Quale tra questi è un esempio di livello di astrazione più alto?

- A. Automobile
- B. Moto
- C. Veicolo
- D. Ruota

Quale caratteristica dovrebbe avere una buona astrazione?

- A. Molti dettagli inutili
- B. Solo elementi rilevanti
- C. Tutti gli attributi possibili
- D. Nessun metodo

Quali strumenti Java supportano l'astrazione?

- A. Solo le classi astratte
- B. Solo le interfacce
- C. Solo i package
- D. Classi, package, classi astratte e interfacce

Esercizi

Esercizio 1

Individuare gli attributi rilevanti per una classe `Libro` .

Esercizio 2

Individuare gli attributi rilevanti per una classe `Automobile` .

Esercizio 3

Eliminare gli attributi inutili dalla seguente classe:

```
class Studente {  
    String nome;  
    String cognome;  
    String colorePenna;  
    String marcaZaino;  
    String matricola;  
}
```

Esercizio 4

Descrivere tre livelli di astrazione per il concetto di veicolo.

Esercizio 5

Spiegare con parole proprie la differenza tra astrazione e incapsulamento.

◆ Associazione in Java

L'**associazione** è una relazione tra due classi che collaborano tra loro mantenendo però la propria indipendenza.

In una associazione, gli oggetti possono conoscersi e interagire, ma ciascuno continua ad avere un proprio ciclo di vita.

◆ **Associazione nel mondo reale**

Molte relazioni del mondo reale sono associazioni.

Esempi:

Studente ↔ Corso

Medico ↔ Paziente

Docente ↔ Classe

Cliente ↔ Ordine

Gli oggetti collaborano tra loro ma possono esistere indipendentemente.

◆ **Caratteristiche dell'associazione**

- le classi sono indipendenti;
- non esiste un proprietario della relazione;
- gli oggetti hanno cicli di vita separati;
- la distruzione di un oggetto non comporta la distruzione dell'altro.

◆ Associazione uno a uno (1:1)

Una persona possiede una patente.

```
class Patente {  
    String numero;  
}  
  
class Persona {  
    String nome;  
    Patente patente;  
}
```

Schema:

Persona — Patente
1 1

◆ Associazione uno a molti (1:N)

Un docente insegna a più studenti.

```
class Docente {  
    String nome;  
}  
  
class Studente {  
    String nome;  
    Docente docente;  
}
```

Schema:

Docente ——— Studente
1 N

Più studenti possono essere associati allo stesso docente.

◆ Associazione molti a molti (N:M)

Uno studente può frequentare più corsi.

Un corso può essere frequentato da più studenti.

Studente ————— Corso
N M

Questo tipo di relazione è molto comune nei database relazionali.

◆ Associazione unidirezionale

Una classe conosce l'altra.

```
class Studente {  
    private Corso corso;  
}
```

Schema:

Studente  Corso

Lo studente conosce il corso.

Il corso non conosce lo studente.

◆ Associazione bidirezionale

Entrambe le classi si conoscono.

Studiante ↔ Corso

Entrambi gli oggetti possono collaborare direttamente.

◆ Esempio completo

```
class Docente {  
    private String nome;  
  
    public Docente(String nome) {  
        this.nome = nome;  
    }  
  
    public String getNome() {  
        return nome;  
    }  
}  
---  
  
class Studente {  
    private String nome;  
    private Docente tutor;  
  
    public Studente(String nome, Docente tutor) {  
        this.nome = nome;  
        this.tutor = tutor;  
    }  
  
    public void mostraTutor() {  
        System.out.println(  
            nome + " ha come tutor " + tutor.getNome()  
        );  
    }  
}
```

Utilizzo:

```
Docente docente = new Docente("Rossi");  
  
Studente studente =  
    new Studente("Mario", docente);  
  
studente.mostraTutor();
```

Output:

```
Mario ha come tutor Rossi
```

◆ Associazione, Aggregazione e Composizione

Le tre relazioni sono spesso confuse.

Associazione

|

└─ collaborazione

Aggregazione

|

└─ contiene

Composizione

|

└─ possiede completamente

Nei prossimi capitoli vedremo le differenze nel dettaglio.

◆ Buone pratiche

- utilizzare l'associazione quando due oggetti collaborano;
- evitare dipendenze inutili;
- mantenere indipendenti i cicli di vita degli oggetti;
- scegliere correttamente la molteplicità della relazione.

Riepilogo

- L'associazione collega due classi indipendenti.
- Gli oggetti mantengono il proprio ciclo di vita.
- Può essere unidirezionale o bidirezionale.
- Può essere 1:1, 1:N oppure N:M.
- È la forma più generale di relazione tra oggetti.

Domande

Che cos'è un'associazione?

- A. Una forma di ereditarietà
- B. Una relazione tra classi indipendenti
- C. Un tipo di costruttore
- D. Un modificatore di accesso

Nell'associazione gli oggetti:

- A. Devono avere lo stesso ciclo di vita
- B. Sono indipendenti
- C. Sono sempre contenuti uno nell'altro
- D. Devono appartenere allo stesso package

Quale relazione rappresenta un docente che segue molti studenti?

- A. 1:1
- B. 1:N
- C. N:1
- D. N:M

Che cosa accade se un oggetto associato viene distrutto?

- A. Tutti gli altri vengono distrutti
- B. La JVM termina
- C. Gli altri possono continuare a esistere
- D. Si genera un errore

Quale simbolo UML rappresenta normalmente un'associazione?

- A. Triangolo
- B. Rombo pieno
- C. Linea semplice
- D. Cerchio

Che cos'è un'associazione unidirezionale?

- A. Entrambi gli oggetti si conoscono
- B. Nessun oggetto conosce l'altro
- C. Un oggetto conosce l'altro
- D. Due oggetti condividono memoria

Che cos'è un'associazione bidirezionale?

- A. Entrambi gli oggetti si conoscono
- B. Gli oggetti sono identici
- C. Gli oggetti vengono copiati
- D. Gli oggetti condividono attributi

Quale tra queste è una tipica associazione?

- A. Studente ↔ Corso
- B. Automobile ↔ Motore
- C. Casa ↔ Stanza
- D. Classe ↔ Interfaccia

Quale relazione verrà studiata nel capitolo successivo?

- A. Casting
- B. Overloading
- C. Aggregazione
- D. Stream API

L'associazione è:

- A. La relazione più generale tra oggetti
- B. Un tipo di costruttore
- C. Una forma di polimorfismo
- D. Un package Java

Esercizi

Esercizio 1

Individuare tre esempi di associazione nel mondo reale.

Esercizio 2

Disegnare una relazione 1:1 tra Persona e Patente.

Esercizio 3

Disegnare una relazione 1:N tra Docente e Studente.

Esercizio 4

Progettare una relazione N:M tra Studente e Corso.

Esercizio 5

Spiegare la differenza tra associazione unidirezionale e bidirezionale.

◆ **Aggregazione in Java**

L'**aggregazione** è una particolare forma di associazione.

Rappresenta una relazione di tipo:

HAS - A

ovvero:

ha un

Un oggetto contiene o utilizza un altro oggetto, ma entrambi possono continuare a esistere indipendentemente.

◆ **Aggregazione nel mondo reale**

Esempi comuni:

Squadra → Giocatore

Scuola → Studente

Biblioteca → Libro

Dipartimento → Docente

La squadra possiede dei giocatori.

Tuttavia un giocatore continua ad esistere anche se lascia la squadra.

◆ **Caratteristiche dell'aggregazione**

- relazione HAS-A;
- caso particolare di associazione;
- esiste un contenitore logico;
- gli oggetti mantengono un ciclo di vita indipendente;
- la distruzione del contenitore non distrugge automaticamente gli oggetti contenuti.

◆ **Associazione vs Aggregazione**

Associazione

Studiante ↔ Corso

I due oggetti collaborano.

Nessuno dei due contiene l'altro.

Aggregazione

Squadra $\longrightarrow$ Giocatore

La squadra possiede giocatori.

Il giocatore continua però ad esistere autonomamente.

◆ Esempio semplice

```
class Motore {  
    private String modello;  
  
    public Motore(String modello) {  
        this.modello = modello;  
    }  
  
    public String getModello() {  
        return modello;  
    }  
}  
---  
  
class Automobile {  
    private String marca;  
    private Motore motore;  
  
    public Automobile(String marca, Motore motore) {  
        this.marca = marca;  
        this.motore = motore;  
    }  
  
    public void mostraInfo() {  
        System.out.println(  
            marca + " - " + motore.getModello()  
        );  
    }  
}
```

Utilizzo:

```
Motore motore = new Motore("1.6 Diesel");  
  
Automobile auto =  
    new Automobile("Fiat", motore);  
  
auto.mostraInfo();
```

Output:

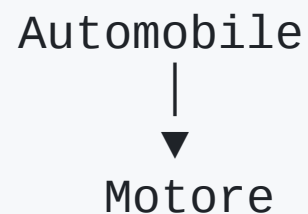
```
Fiat - 1.6 Diesel
```

◆ Indipendenza del ciclo di vita

Il punto fondamentale dell'aggregazione è che gli oggetti possono vivere separatamente.

```
Motore motore = new Motore("1.6 Diesel");  
  
Automobile auto =  
    new Automobile("Fiat", motore);
```

Schema:



Se l'automobile viene eliminata:

Automobile ✕

il motore può continuare ad esistere:

Motore ✓

◆ Rappresentazione UML

In UML l'aggregazione è rappresentata con un rombo vuoto.

Squadra ◇—— Giocatore

Il rombo viene posto sul lato dell'oggetto aggregatore.

◆ **Aggregazione e Composizione**

L'aggregazione viene spesso confusa con la composizione.

Aggregazione

|
├─ gli oggetti possono esistere separatamente

Composizione

|
└─ gli oggetti dipendono dal contenitore

La composizione verrà approfondita nel prossimo capitolo.

◆ Buone pratiche

- usare l'aggregazione quando esiste una relazione HAS-A;
- mantenere indipendenti i cicli di vita;
- evitare di usare la composizione quando non necessaria;
- modellare correttamente le relazioni del dominio.

Riepilogo

- L'aggregazione è una forma particolare di associazione.
- Rappresenta una relazione HAS-A.
- Esiste un contenitore logico.
- Gli oggetti mantengono un ciclo di vita indipendente.
- In UML è rappresentata da un rombo vuoto.
- È meno forte della composizione.

Domande

Che cos'è l'aggregazione?

- A. Una forma di ereditarietà
- B. Una forma particolare di associazione
- C. Un costruttore speciale
- D. Un'interfaccia

Quale relazione rappresenta l'aggregazione?

- A. IS-A
- B. USES-A
- C. HAS-A
- D. EXTENDS-A

Quale caratteristica distingue l'aggregazione?

- A. Gli oggetti condividono memoria
- B. Gli oggetti hanno ciclo di vita indipendente
- C. Gli oggetti devono appartenere allo stesso package
- D. Gli oggetti sono sempre statici

Quale tra questi è un esempio di aggregazione?

- A. Squadra → Giocatore
- B. Studente → Corso
- C. Persona → Classe Astratta
- D. Metodo → Variabile

Nell'aggregazione esiste:

- A. Un contenitore logico
- B. Una sottoclasse
- C. Un'interfaccia obbligatoria
- D. Un metodo astratto

Se viene eliminato l'oggetto aggregatore:

- A. Gli oggetti contenuti vengono sempre eliminati
- B. Il programma termina
- C. Gli oggetti contenuti possono continuare ad esistere
- D. Si genera un'eccezione

Quale simbolo UML rappresenta l'aggregazione?

- A. Triangolo
- B. Cerchio
- C. Rombo pieno
- D. Rombo vuoto

L'aggregazione è:

- A. Più forte dell'associazione
- B. Uguale alla composizione
- C. Una forma di polimorfismo
- D. Un modificatore di accesso

Quale capitolo approfondisce la relazione più forte dell'aggregazione?

- A. Ereditarietà
- B. Polimorfismo
- C. Composizione
- D. Enum

Quale affermazione è corretta?

- A. Un motore può esistere senza automobile
- B. Un motore deve sempre appartenere a una automobile
- C. Un motore è una sottoclasse di automobile
- D. Un motore è un'interfaccia

Esercizi

Esercizio 1

Individuare tre esempi di aggregazione nel mondo reale.

Esercizio 2

Disegnare una relazione di aggregazione tra Scuola e Studente.

Esercizio 3

Creare una classe `Libro` e una classe `Biblioteca` in relazione di aggregazione.

Esercizio 4

Spiegare la differenza tra associazione e aggregazione.

Esercizio 5

Spiegare perché la relazione Automobile → Motore può essere considerata un'aggregazione.

◆ Composizione in Java

La **composizione** è una forma particolare di aggregazione.

Rappresenta una relazione molto forte tra due oggetti:

HAS - A
PART - OF

ovvero:

ha un
fa parte di

L'oggetto contenuto non può esistere indipendentemente dal contenitore.

◆ Composizione nel mondo reale

Esempi tipici:

Casa → Stanza

Ordine → RigaOrdine

Libro → Pagina

Computer → SchedaMadre

Se il contenitore scompare, anche le sue parti perdono significato.

◆ **Caratteristiche della composizione**

- relazione HAS-A molto forte;
- relazione PART-OF;
- esiste un proprietario reale;
- il ciclo di vita degli oggetti è condiviso;
- le parti vengono spesso create dal contenitore;
- le parti non vengono condivise con altri oggetti.

◆ **Aggregazione vs Composizione**

Aggregazione

Squadra → Giocatore

Il giocatore continua ad esistere anche se la squadra viene sciolta.

Composizione

Casa → Stanza

Una stanza non può esistere indipendentemente dalla casa che la contiene.

◆ Esempio semplice

```
class Stanza {  
    private String nome;  
  
    public Stanza(String nome) {  
        this.nome = nome;  
    }  
  
    public String getNome() {  
        return nome;  
    }  
}
```

```
class Casa {  
  
    private Stanza soggiorno;  
    private Stanza cucina;  
  
    public Casa() {  
        soggiorno = new Stanza("Soggiorno");  
        cucina = new Stanza("Cucina");  
    }  
  
    public void mostraStanze() {  
  
        System.out.println(  
            soggiorno.getNome()  
        );  
        ---  
  
        System.out.println(  
            cucina.getNome()  
        );  
    }  
}
```

Utilizzo:

Output:

```
Soggiorno  
Cucina
```

◆ Dipendenza del ciclo di vita

La caratteristica principale della composizione è la dipendenza del ciclo di vita.

Schema:



Se la casa viene eliminata:

Casa ✕

anche le stanze cessano di esistere:

Stanza ✕
Stanza ✕

◆ Creazione interna

Nella composizione il contenitore crea spesso direttamente gli oggetti che possiede.

```
public Casa() {  
    soggiorno =  
        new Stanza("Soggiorno");  
  
    cucina =  
        new Stanza("Cucina");  
}
```

Questo rafforza il legame tra contenitore e parti.

◆ Rappresentazione UML

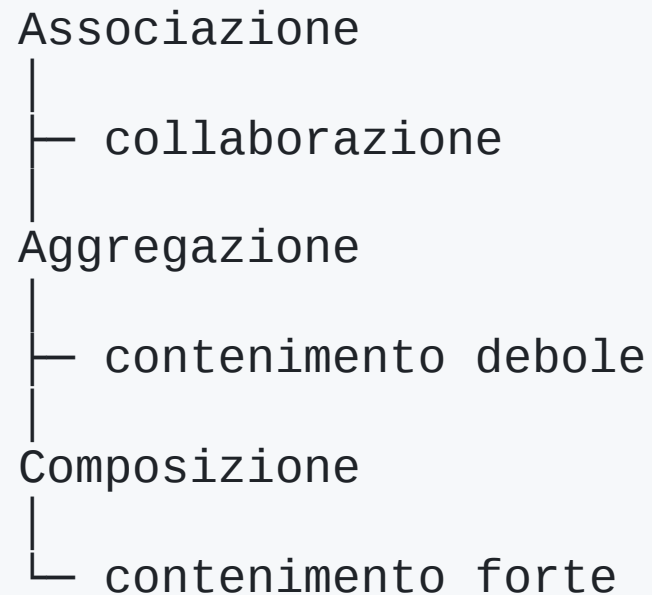
In UML la composizione viene rappresentata tramite un rombo pieno.

Casa ◆—— Stanza

Il rombo viene posto sul lato del contenitore.

◆ Associazione, Aggregazione e Composizione

Possiamo vedere queste relazioni come una scala crescente.



◆ Quando usare la composizione

La composizione è appropriata quando:

- una parte appartiene completamente al contenitore;
- la parte non viene condivisa;
- il ciclo di vita dipende dal contenitore;
- esiste una relazione PART-OF evidente.

◆ Buone pratiche

- preferire la composizione quando esiste una vera relazione PART-OF;
- evitare di condividere le parti tra più contenitori;
- creare gli oggetti interni all'interno del contenitore;
- distinguere correttamente aggregazione e composizione.

Riepilogo

- La composizione è una forma forte di aggregazione.
- Rappresenta una relazione HAS-A e PART-OF.
- Gli oggetti condividono il ciclo di vita.
- Le parti non esistono indipendentemente dal contenitore.
- In UML è rappresentata da un rombo pieno.
- È la relazione più forte tra oggetti.

Domande

Che cos'è la composizione?

- A. Una forma di ereditarietà
- B. Una forma forte di aggregazione
- C. Un costruttore speciale
- D. Un modificatore di accesso

Quale relazione rappresenta la composizione?

- A. IS-A
- B. USES-A
- C. HAS-A / PART-OF
- D. IMPLEMENTS-A

Quale caratteristica distingue la composizione?

- A. Gli oggetti hanno ciclo di vita indipendente
- B. Gli oggetti condividono il ciclo di vita
- C. Gli oggetti devono essere statici
- D. Gli oggetti appartengono allo stesso package

Quale tra questi è un esempio di composizione?

- A. Studente → Corso
- B. Squadra → Giocatore
- C. Casa → Stanza
- D. Docente → Studente

Nella composizione esiste:

- A. Un proprietario reale delle parti
- B. Una sottoclasse
- C. Un'interfaccia obbligatoria
- D. Un metodo astratto

Se il contenitore viene eliminato:

- A. Le parti possono continuare ad esistere
- B. Le parti vengono eliminate
- C. La JVM termina
- D. Si genera un errore

Quale simbolo UML rappresenta la composizione?

- A. Triangolo
- B. Cerchio
- C. Rombo vuoto
- D. Rombo pieno

La composizione è:

- A. Più debole dell'aggregazione
- B. Uguale all'associazione
- C. Più forte dell'aggregazione
- D. Un tipo di polimorfismo

Nella composizione le parti sono normalmente:

- A. Condivise tra più contenitori
- B. Statiche
- C. Indipendenti
- D. Esclusive del contenitore

Quale relazione è la più forte tra oggetti?

- A. Associazione
- B. Aggregazione
- C. Composizione
- D. Overloading

Esercizi

Esercizio 1

Individuare tre esempi di composizione nel mondo reale.

Esercizio 2

Disegnare una relazione di composizione tra Casa e Stanza.

Esercizio 3

Creare una classe `Libro` contenente alcune `Pagina` .

Esercizio 4

Spiegare la differenza tra aggregazione e composizione.

Esercizio 5

Classificare le seguenti relazioni come associazione, aggregazione o composizione:

- `Studiante` → `Corso`
- `Squadra` → `Giocatore`
- `Casa` → `Stanza`
- `Biblioteca` → `Libro`
- `Ordine` → `RigaOrdine`

◆ Ereditarietà in Java

L'**ereditarietà** è uno dei principi fondamentali della programmazione orientata agli oggetti.

Permette di creare nuove classi a partire da classi esistenti, riutilizzandone attributi e metodi.

L'obiettivo principale è evitare duplicazioni di codice e modellare correttamente le relazioni tra oggetti.

◆ Il concetto di IS-A

L'ereditarietà rappresenta una relazione di tipo:

IS-A

ovvero:

è un

Esempi:

```
Automobile è un Veicolo
```

```
---
```

```
Moto è un Veicolo
```

```
Studente è una Persona
```

```
Docente è una Persona
```

◆ Superclasse e sottoclasse

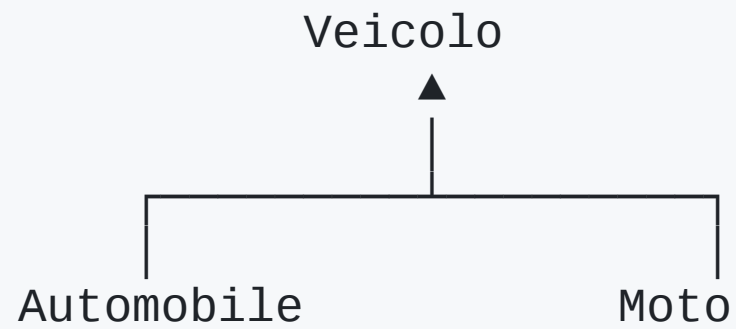
La classe da cui si eredita prende il nome di:

Superclasse
Classe base
Classe padre

La classe che eredita prende il nome di:

Sottoclasse
Classe derivata
Classe figlia

Schema:



◆ La parola chiave extends

In Java l'ereditarietà si realizza tramite la parola chiave:

```
extends
```

Esempio:

```
class Veicolo {  
    String marca;  
  
    public void accendi() {  
        System.out.println("Veicolo acceso");  
    }  
}
```

```
class Automobile extends Veicolo {  
    int numeroPorte;  
}
```

◆ Ereditare attributi e metodi

La sottoclasse eredita automaticamente gli elementi accessibili della superclasse.

```
Automobile auto = new Automobile();  
  
auto.marca = "Fiat";  
auto.accendi();
```

Anche se `marca` e `accendi()` sono definiti nella superclasse.

◆ Estendere una classe

La sottoclasse può aggiungere nuovi attributi e metodi.

```
class Automobile extends Veicolo {  
    int numeroPorte;  
  
    public void apriBagagliaio() {  
        System.out.println("Bagagliaio aperto");  
    }  
}
```

◆ Esempio completo

```
class Persona {  
    String nome;  
  
    public void saluta() {  
        System.out.println("Ciao");  
    }  
}
```

```
class Studente extends Persona {  
    ---  
  
    String matricola;  
}
```

Utilizzo:

```
Studente s = new Studente();
```

```
s.nome = "Mario";  
s.matricola = "A123";
```

◆ Catena di ereditarietà

Una classe può ereditare da un'altra classe che a sua volta eredita da una terza.



Ogni livello eredita le caratteristiche dei livelli superiori.

◆ Cosa non viene ereditato

Non tutto viene ereditato.

Gli elementi dichiarati:

```
private
```

non sono direttamente accessibili nella sottoclasse.

```
class Persona {  
    private String codiceFiscale;  
}
```

La sottoclasse non può accedere direttamente a `codiceFiscale` .

◆ Ereditarietà e riuso del codice

Senza ereditarietà:

```
Persona  
Studente  
Docente
```

potrebbero contenere codice duplicato.

Con l'ereditarietà:

```
Persona  
├──  
│   Studente  
│   Docente
```

il codice comune viene scritto una sola volta.

◆ Quando usare l'ereditarietà

L'ereditarietà è appropriata quando esiste una vera relazione:

IS-A

Ad

esempio:

Studente è una Persona ✓

Automobile è un Veicolo ✓

Mentre:

Motore è una Automobile ✗

Ruota è una Automobile ✗

◆ Buone pratiche

- usare l'ereditarietà solo quando esiste una vera relazione IS-A;
- evitare gerarchie troppo profonde;
- privilegiare il riuso del codice;
- distinguere correttamente tra IS-A e HAS-A.

Riepilogo

- L'ereditarietà permette di riutilizzare il codice.
- Si realizza tramite la parola chiave `extends`.
- La sottoclasse eredita attributi e metodi dalla superclasse.
- Rappresenta una relazione di tipo IS-A.
- Favorisce il riuso e la manutenibilità del codice.
- Deve essere utilizzata solo quando esiste una reale specializzazione.

Domande

Che cos'è l'ereditarietà?

- A. Una forma di associazione
- B. Un meccanismo di riutilizzo del codice
- C. Un modificatore di accesso
- D. Un tipo di costruttore

Quale parola chiave si usa per ereditare una classe?

- A. implements
- B. inherits
- C. extends
- D. super

L'ereditarietà rappresenta una relazione:

- A. HAS-A
- B. USES-A
- C. PART-OF
- D. IS-A

Come si chiama la classe da cui si eredita?

- A. Sottoclasse
- B. Interfaccia
- C. Superclasse
- D. Record

Come si chiama la classe che eredita?

- A. Sottoclasse
- B. Package
- C. Modulo
- D. Enum

Quale relazione è corretta?

- A. Motore è una Automobile
- B. Studente è una Persona
- C. Ruota è una Automobile
- D. Libro è una Biblioteca

Una sottoclasse eredita:

- A. Solo i costruttori
- B. Attributi e metodi accessibili
- C. Solo gli attributi privati
- D. Solo i metodi statici

Quale modificatore impedisce l'accesso diretto dalla sottoclasse?

- A. public
- B. protected
- C. static
- D. private

Quale tra queste è una buona applicazione dell'ereditarietà?

- A. Automobile → Motore
- B. Casa → Stanza
- C. Veicolo → Automobile
- D. Biblioteca → Libro

L'obiettivo principale dell'ereditarietà è:

- A. Ridurre la duplicazione del codice
- B. Velocizzare la compilazione
- C. Eliminare gli oggetti
- D. Evitare i package

Esercizi

Esercizio 1

Creare una classe `Persona` e una classe `Studente` che erediti da essa.

Esercizio 2

Creare una gerarchia:

```
Veicolo
├── Automobile
└── Moto
```

Esercizio 3

Indicare quali delle seguenti relazioni sono IS-A:

- `Studiante` → `Persona`
- `Automobile` → `Veicolo`
- `Motore` → `Automobile`
- `Docente` → `Persona`

Esercizio 4

Disegnare il diagramma UML di una gerarchia composta da:

```
Animale
├── Cane
├── Gatto
└── Uccello
```

Esercizio 5

Spiegare la differenza tra:

IS-A

e

HAS-A

◆ Polimorfismo in Java

La parola **polimorfismo** deriva dal greco e significa:

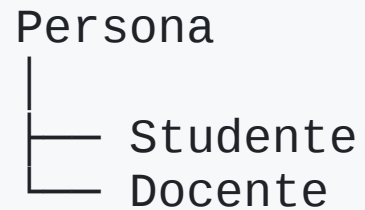
molte forme

In programmazione orientata agli oggetti, il polimorfismo è la capacità di utilizzare oggetti diversi attraverso un riferimento comune.

Grazie al polimorfismo, uno stesso metodo può produrre comportamenti differenti a seconda dell'oggetto che lo esegue.

◆ Perché serve il polimorfismo

Nel capitolo precedente abbiamo visto l'ereditarietà.



Sia `Studente` che `Docente` sono anche delle `Persona`.

Per questo motivo possiamo scrivere:

```
Persona p = new Studente();
```

oppure:

```
Persona p = new Docente();
```

La variabile ha sempre tipo `Persona`, ma può riferirsi a oggetti differenti.

◆ Esempio base

```
class Persona {  
    public void presentati() {  
        System.out.println("Sono una persona");  
    }  
}
```

```
class Studente extends Persona {  
    @Override  
    public void presentati() {  
        System.out.println("Sono uno studente");  
    }  
}
```

Utilizzo:

```
Persona p = new Studente();  
p.presentati();
```

Output:

```
Sono uno studente
```

Viene eseguita la versione del metodo appartenente all'oggetto reale.

◆ Tipo del riferimento e tipo dell'oggetto

È importante distinguere:

```
Persona p = new Studente();
```

Tipo del riferimento → Persona

Tipo dell'oggetto → Studente

Il compilatore controlla il tipo del riferimento.

La JVM esegue il metodo dell'oggetto reale.

◆ Dynamic Binding

La scelta del metodo corretto avviene durante l'esecuzione del programma.

Questo meccanismo prende il nome di:

```
Dynamic Binding  
Late Binding
```

Schema:

```
Persona p  
  |  
  ▼  
Studiante  
  |  
  ▼  
presentati()
```

La JVM individua il metodo corretto in base all'oggetto effettivo.

◆ Più oggetti, stesso riferimento

```
Persona[] persone = {  
    new Studente(),  
    new Docente()  
};
```

```
for (Persona p : persone) {  
    p.presentati();  
}
```


Output:

```
Sono uno studente  
Sono un docente
```

Lo stesso codice funziona per oggetti differenti.

◆ Override dei metodi

Per personalizzare il comportamento di un metodo ereditato si utilizza l'override.

```
@Override  
public void presentati() {  
    System.out.println("Sono uno studente");  
}
```

L'annotazione:

```
@Override
```

non è obbligatoria ma è fortemente consigliata.

◆ **Vantaggi del polimorfismo**

- codice più flessibile;
- maggiore riuso;
- minore accoppiamento;
- estensione più semplice delle applicazioni;
- codice più facile da mantenere.

◆ Polimorfismo e Overloading

Il termine polimorfismo viene talvolta utilizzato anche per indicare l'overloading.

```
class Operazione {  
    int raddoppia(int n) {  
        return n * 2;  
    }  
  
    String raddoppia(String s) {  
        return s + s;  
    }  
}
```

In questo caso il compilatore sceglie il metodo corretto in base ai parametri.

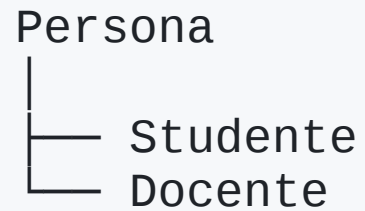
Questo meccanismo viene chiamato:

Polimorfismo statico
Overloading

Tuttavia, quando si parla di polimorfismo in OOP, normalmente ci si riferisce al polimorfismo tramite ereditarietà.

◆ Polimorfismo ed Ereditarietà

L'ereditarietà crea la gerarchia:



Il polimorfismo permette di utilizzare tali oggetti tramite un riferimento comune.

I due concetti lavorano insieme.

◆ Buone pratiche

- utilizzare il tipo più generale possibile;
- progettare gerarchie semplici;
- usare l'override per personalizzare i comportamenti;
- evitare cast non necessari.

Riepilogo

- Il polimorfismo significa "molte forme".
- Permette di utilizzare oggetti diversi tramite un riferimento comune.
- Si basa sull'ereditarietà.
- La JVM sceglie il metodo corretto durante l'esecuzione.
- Questo meccanismo è chiamato Dynamic Binding.
- L'override consente di specializzare il comportamento dei metodi.

Domande

Che cosa significa polimorfismo?

- A. Molte classi
- B. Molte forme
- C. Molti package
- D. Molti attributi

Quale relazione rende possibile il polimorfismo?

- A. Aggregazione
- B. Composizione
- C. Ereditarietà
- D. Incapsulamento

Quale istruzione è valida?

- A. `Studiante s = new Persona();`
- B. `Persona p = new Studiante();`
- C. `Persona p = new String();`
- D. `Studiante s = new Docente();`

In:

```
Persona p = new Studente();
```

qual è il tipo del riferimento?

- A. Studente
- B. Object
- C. Persona
- D. Dipende dalla JVM

Qual è il tipo dell'oggetto creato?

- A. Persona
- B. Object
- C. Variabile
- D. Studente

Chi decide quale metodo eseguire?

- A. Il package
- B. La JVM durante l'esecuzione
- C. Il compilatore sempre
- D. Il Garbage Collector

Come si chiama la ridefinizione di un metodo ereditato?

- A. Overloading
- B. Casting
- C. Override
- D. Boxing

Quale annotazione viene normalmente utilizzata?

- A. @Static
- B. @Override
- C. @Extend
- D. @Inherited

Quale vantaggio offre il polimorfismo?

- A. Riduce la flessibilità
- B. Elimina le classi
- C. Permette codice più generico
- D. Elimina i costruttori

Quale tipo di polimorfismo utilizza l'overloading?

- A. Dinamico
- B. Statico
- C. Parametrico
- D. Multiplo

Esercizi

Esercizio 1

Creare una gerarchia:

```
Persona
├── Studente
└── Docente
```

con un metodo `presentati()` ridefinito.

Esercizio 2

Creare un array di `Persona` contenente studenti e docenti.

Esercizio 3

Realizzare una gerarchia:

```
Animale
├── Cane
├── Gatto
└── Uccello
```

e ridefinire il metodo `verso()` .

Esercizio 4

Spiegare la differenza tra overloading e override.

Esercizio 5

Spiegare con parole proprie il significato di:

```
Persona p = new Studente();
```

◆ Il modificatore static in Java

La parola chiave:

```
static
```

permette di dichiarare attributi e metodi che appartengono alla **classe** anziché ai singoli oggetti.

Grazie a `static`, una risorsa viene condivisa da tutte le istanze della classe.

◆ **Livello di classe e livello di istanza**

In Java esistono due livelli differenti.

Livello di istanza

Ogni oggetto possiede la propria copia dell'attributo.

```
class Persona {  
    String nome;  
}
```

Schema:

```
Persona p1 → nome = "Luca"  
Persona p2 → nome = "Anna"
```

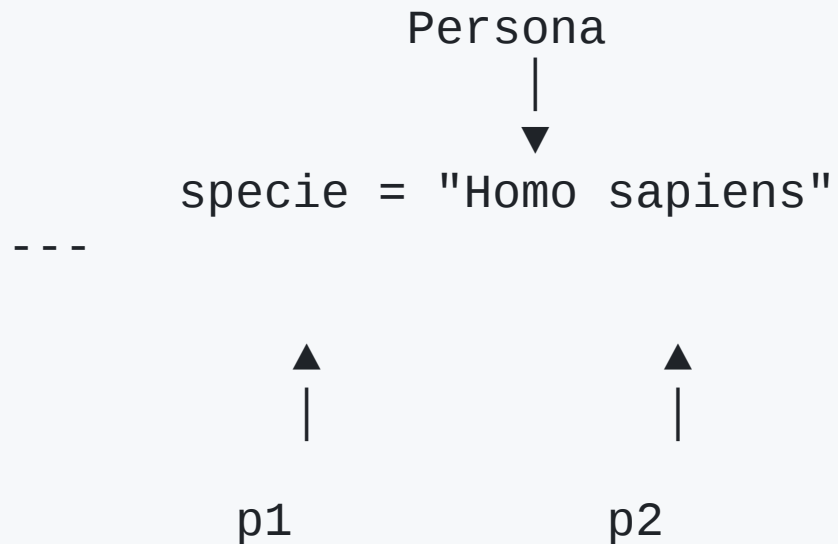
Ogni oggetto ha un valore indipendente.

Livello di classe

L'attributo appartiene alla classe.

```
class Persona {  
    static String specie = "Homo sapiens";  
}
```

Schema:



Esiste una sola copia dell'attributo

◆ **Attributi static**

Gli attributi static rappresentano informazioni condivise da tutti gli oggetti.

```
class Automobile {  
    static int numeroRuote = 4;  
}
```

Utilizzo:

```
System.out.println(  
    Automobile.numeroRuote  
);
```

Output:

4

◆ Contatore di oggetti

Un utilizzo molto comune consiste nel contare le istanze create.

```
class Persona {  
  
    static int contatore = 0;  
  
    public Persona() {  
        contatore++;  
    }  
}
```

Utilizzo:

```
new Persona();  
new Persona();  
new Persona();  
  
System.out.println(  
    Persona.contatore  
);
```

◆ Metodi static

I metodi static appartengono alla classe.

```
class Calcolatrice {  
    public static int somma(  
        int a,  
        int b  
    ) {  
        return a + b;  
    }  
}
```

Utilizzo:

```
int risultato =  
    Calcolatrice.somma(3, 5);
```

Non è necessario creare oggetti.

◆ Esempi dalla libreria Java

Classe Math

```
double radice =  
    Math.sqrt(25);
```

Classe Integer

```
int n =  
    Integer.parseInt("123");
```

Classe String

```
String s =  
    String.valueOf(123);
```

Tutti questi sono esempi di metodi static.

◆ Metodo main

Il metodo principale di un programma Java è static.

```
public static void main(  
    String[] args  
)
```

Questo permette alla JVM di eseguirlo senza creare prima un oggetto.

◆ Limitazioni dei metodi static

Un metodo static non può accedere direttamente agli attributi di istanza.

Esempio errato:

```
class Persona {  
    String nome;  
  
    public static void stampa() {  
        System.out.println(nome);  
    }  
}
```

Errore:

```
Cannot make a static reference  
to the non-static field
```

◆ Perché accade?

Perché un metodo static può essere eseguito anche quando non esistono oggetti.

```
Persona.stampa();
```

In questo caso non esiste alcuna istanza da cui recuperare `nome`.

◆ Static e oggetti

È tecnicamente possibile scrivere:

```
Persona p = new Persona();  
p.contatore++;
```

ma è sconsigliato.

Meglio:

```
Persona.contatore++;
```

perché rende evidente che l'attributo appartiene alla classe.

◆ Quando usare static

Utilizzare `static` quando:

- il dato è condiviso da tutte le istanze;
- il metodo non dipende dallo stato dell'oggetto;
- si vuole creare una utility class;
- si desidera mantenere un contatore globale.

◆ Quando NON usare static

Evitare `static` quando:

- il dato appartiene al singolo oggetto;
- il metodo utilizza attributi dell'istanza;
- si vuole sfruttare il polimorfismo.

Riepilogo

- Gli elementi static appartengono alla classe.
- Gli elementi non static appartengono agli oggetti.
- Esiste una sola copia di un attributo static.
- I metodi static possono essere invocati senza creare oggetti.
- Il metodo `main()` è static.
- Un metodo static non può accedere direttamente agli attributi di istanza.

Domande

Che cosa indica la parola chiave `static` ?

- A. Che l'elemento appartiene all'oggetto
- B. Che l'elemento appartiene alla classe
- C. Che l'elemento è privato
- D. Che l'elemento è astratto

Quante copie esistono di un attributo static?

- A. Una per ogni oggetto
- B. Una per ogni package
- C. Una sola
- D. Dipende dalla JVM

Come si accede normalmente a un attributo static?

- A. tramite this
- B. tramite il nome della classe
- C. tramite new
- D. tramite super

Quale dei seguenti è un metodo static della libreria Java?

- A. `String.length()`
- B. `Math.sqrt()`
- C. `ArrayList.add()`
- D. `Scanner.nextLine()`

Perché il metodo main è static?

- A. Per migliorare le prestazioni
- B. Per essere eseguito senza creare oggetti
- C. Perché è obbligatorio usare static
- D. Perché è privato

Un metodo static può accedere direttamente a un attributo non static?

- A. Sempre
- B. Mai
- C. Solo tramite un oggetto
- D. Solo nelle sottoclassi

Quale utilizzo è tipico di un attributo static?

- A. Nome di una persona
- B. Età di uno studente
- C. Contatore delle istanze create
- D. Saldo di un conto corrente

Quale classe della libreria Java contiene molti metodi static?

- A. Scanner
- B. Math
- C. ArrayList
- D. JFrame

Quale affermazione è corretta?

- A. Gli attributi static appartengono agli oggetti
- B. I metodi static richiedono sempre un oggetto
- C. Gli attributi static appartengono alla classe
- D. Static e final sono sinonimi

Quando è opportuno usare static?

- A. Quando il comportamento è indipendente dagli oggetti
- B. Quando si vuole fare ereditarietà
- C. Quando si usa il polimorfismo
- D. Quando si vuole nascondere un attributo

Esercizi

Esercizio 1

Creare una classe `Studente` con un contatore static delle istanze create.

Esercizio 2

Creare una classe `Calcolatrice` con metodi static:

- somma
- sottrai
- moltiplica
- dividi

Esercizio 3

Indicare quali elementi dovrebbero essere static:

- nomeStudente
- PI_GRECO
- numeroRuote
- contatoreUtenti

Esercizio 4

Spiegare la differenza tra:

```
String nome;
```

e

```
static String nome;
```

Esercizio 5

Spiegare perché il metodo:

```
public static void main(String[] args)
```

deve essere dichiarato static.

◆ Classi Astratte in Java

Una **classe astratta** è una classe pensata per essere utilizzata come base di altre classi.

Può contenere:

- attributi;
- metodi concreti (con implementazione);
- metodi astratti (senza implementazione).

Una classe astratta rappresenta spesso un concetto troppo generico per poter creare oggetti direttamente.

◆ **Classe concreta e classe astratta**

Classe concreta

Una classe concreta può essere istanziata.

```
class Persona {  
    String nome;  
}
```

Utilizzo:

```
Persona p = new Persona();
```

Classe astratta

Una classe astratta non può essere istanziata.

```
abstract class Persona {  
}
```

Tentare di creare un oggetto genera un errore:

```
Persona p = new Persona(); // ERRORE
```

◆ La parola chiave **abstract**

Per dichiarare una classe astratta si utilizza:

```
abstract
```

Esempio:

```
abstract class Forma {  
}
```

La classe può essere utilizzata soltanto come superclasse.

◆ Metodi astratti

Un metodo astratto definisce una firma ma non un'implementazione.

```
abstract class Forma {  
    public abstract void disegna();  
}
```

Caratteristiche:

- non contiene corpo;
- termina con `;`;
- deve essere implementato dalle sottoclassi concrete.

◆ Perché usare un metodo astratto

A volte sappiamo che tutte le sottoclassi dovranno avere un certo comportamento.

Ad

esempio:

```
Forma
├── Cerchio
├── Rettangolo
└── Triangolo
```

Tutte le forme devono poter essere disegnate.

Possiamo quindi dichiarare:

```
public abstract void disegna();
```

senza specificare come.

◆ Esempio completo

```
abstract class Forma {  
    public abstract void disegna();  
}
```

```
class Cerchio extends Forma {  
    @Override  
    public void disegna() {  
        System.out.println("Disegno un cerchio");  
    }  
}
```

Utilizzo:

```
Forma f = new Cerchio();  
f.disegna();
```

Output:

```
Disegno un cerchio
```

◆ Classi parzialmente astratte

Una classe astratta può contenere anche metodi normali.

```
abstract class Forma {  
    public void saluta() {  
        System.out.println("Sono una forma");  
    }  
    public abstract void disegna();  
}
```

In questo caso:

- `saluta()` è già implementato;
- `disegna()` dovrà essere implementato dalle sottoclassi.

◆ Obblighi delle sottoclassi

Se una sottoclasse eredita una classe astratta:

```
abstract class Forma {  
    public abstract void disegna();  
}
```

deve implementare tutti i metodi astratti.

```
class Cerchio extends Forma {  
    @Override  
    public void disegna() {  
        System.out.println("Cerchio");  
    }  
}
```

◆ Sottoclassi astratte

Una sottoclasse può decidere di non implementare i metodi astratti.

In tal caso deve essere anch'essa dichiarata astratta.

```
abstract class Forma {  
    public abstract void disegna();  
}
```

```
abstract class FormaGeometrica  
    extends Forma {  
}
```

◆ Classi astratte e polimorfismo

Le classi astratte vengono spesso utilizzate insieme al polimorfismo.

```
Forma f1 = new Cerchio();  
Forma f2 = new Rettangolo();  
Forma f3 = new Triangolo();
```

Tutte le variabili hanno tipo `Forma` , ma gli oggetti sono differenti.

```
f1.disegna();  
f2.disegna();  
f3.disegna();
```

La JVM eseguirà la versione corretta del metodo.

◆ Quando usare una classe astratta

Una classe astratta è utile quando:

- esiste una relazione IS-A;
- si vuole fornire comportamento comune;
- si desidera forzare alcuni metodi nelle sottoclassi;
- la superclasse è troppo generica per essere istanziata.

◆ Classi astratte e interfacce

Le classi astratte e le interfacce hanno obiettivi simili:

Definire un contratto comune

Le differenze verranno approfondite nel prossimo capitolo dedicato alle interfacce.

◆ Buone pratiche

- utilizzare classi astratte per rappresentare concetti generali;
- inserire nella superclasse il codice comune;
- usare metodi astratti per i comportamenti obbligatori;
- evitare gerarchie troppo profonde.

✓ Riepilogo

- Una classe astratta non può essere istanziata.
- Si dichiara tramite la parola chiave `abstract`.
- Può contenere metodi concreti e metodi astratti.
- I metodi astratti non hanno implementazione.
- Le sottoclassi concrete devono implementare tutti i metodi astratti.
- Le classi astratte sono spesso utilizzate insieme al polimorfismo.

Domande

Che cos'è una classe astratta?

- A. Una classe che contiene solo attributi
- B. Una classe che non può essere istanziata
- C. Una classe senza metodi
- D. Una sottoclasse speciale

Quale parola chiave si usa per dichiarare una classe astratta?

- A. extends
- B. interface
- C. abstract
- D. virtual

Una classe astratta può essere istanziata?

- A. Sempre
- B. Solo nelle sottoclassi
- C. Solo se ha costruttori
- D. No

Che cos'è un metodo astratto?

- A. Un metodo static
- B. Un metodo senza implementazione
- C. Un costruttore
- D. Un metodo privato

Come termina la dichiarazione di un metodo astratto?

A. Con }

B. Con return

C. Con ;

D. Con :

Una classe astratta può contenere metodi concreti?

- A. No
- B. Solo static
- C. Solo protected
- D. Sì

Cosa deve fare una sottoclasse concreta?

- A. Eliminare i metodi astratti
- B. Implementare i metodi astratti ereditati
- C. Dichiarare tutti i metodi static
- D. Rendere pubblici tutti gli attributi

Se una sottoclasse non implementa i metodi astratti?

- A. Deve essere dichiarata astratta
- B. Diventa final
- C. Diventa statica
- D. Non compila mai

Quale concetto lavora spesso insieme alle classi astratte?

- A. Casting
- B. Polimorfismo
- C. Boxing
- D. Package

Quale gerarchia è un buon candidato per una classe astratta?

A. Forma → Cerchio → Rettangolo

B. Math → PI

C. Scanner → Input

D. String → Char

Esercizi

Esercizio 1

Creare una classe astratta `Animale` con il metodo astratto:

```
public abstract void verso();
```

Esercizio 2

Creare le sottoclassi:

```
Animale
├── Cane
└── Gatto
```

implementando `verso()` .

Esercizio 3

Creare una classe astratta `Forma` con:

- metodo concreto `saluta()`
- metodo astratto `disegna()`

Esercizio 4

Creare una gerarchia:

```
Veicolo
├── Automobile
├── Moto
└── Camion
```

utilizzando una classe astratta come base.

Esercizio 5

Spiegare con parole proprie la differenza tra:

```
class Persona
```

e

```
abstract class Persona
```

◆ Interfacce in Java

Le **interfacce** sono uno degli strumenti più importanti della programmazione orientata agli oggetti.

Un'interfaccia definisce un insieme di operazioni che una classe deve fornire, senza specificarne l'implementazione.

Possiamo considerarla come un **contratto**.

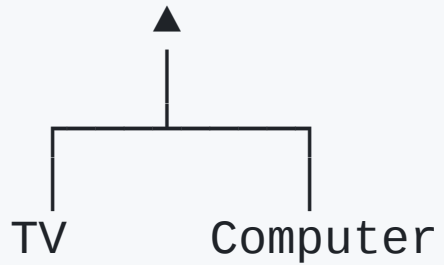
L'interfaccia dice COSA fare.

La classe decide COME farlo.

◆ Il concetto di contratto

Immaginiamo una presa elettrica.

Presa elettrica



Tutti i dispositivi rispettano lo stesso standard.

Non importa come siano costruiti internamente.

Lo stesso principio vale per le interfacce.

◆ Definizione di un'interfaccia

Si utilizza la parola chiave:

```
interface
```

Esempio:

```
public interface Stampabile {  
    void stampa();  
}
```


I metodi dichiarano il comportamento richiesto.

Non viene fornita alcuna implementazione.

◆ Implementazione di un'interfaccia

Una classe implementa un'interfaccia tramite:

```
implements
```

```
public class Documento
    implements Stampabile {

    @Override
    public void stampa() {
        System.out.println("Documento");
    }
}
```

◆ Utilizzo

```
Stampabile s =  
    new Documento();  
  
s.stampa();
```

Output:

```
Documento
```

◆ Interfacce e polimorfismo

Le interfacce vengono utilizzate soprattutto con il polimorfismo.

```
interface Stampabile {  
    void stampa();  
}
```

```
class Documento  
    implements Stampabile {  
  
    public void stampa() {  
        System.out.println("Documento");  
    }  
}
```

```
class Foto
    implements Stampabile {

    public void stampa() {
        System.out.println("Foto");
    }
}
```

Utilizzo:

```
Stampabile s1 =
    new Documento();

---

Stampabile s2 =
    new Foto();
```

Entrambi gli oggetti vengono utilizzati tramite il tipo comune `Stampabile` .

◆ **Interfacce e classi astratte**

Entrambe definiscono un contratto comune.

Classe astratta

```
abstract class Forma {  
    public abstract void disegna();  
}
```

Può contenere:

- attributi;
- metodi concreti;
- metodi astratti.

Interfaccia

```
interface Disegnabile {  
    void disegna();  
}
```

Definisce principalmente comportamenti.

◆ Quando usare una classe astratta

Quando esiste una relazione:

IS-A

Esempio:

```
Persona
├── Studente
└── Docente
```

◆ Quando usare un'interfaccia

Quando vogliamo descrivere una capacità.

```
Stampabile  
Salvabile  
Disegnabile  
Navigabile
```

Esempio:



Documento e Foto non sono lo stesso tipo di oggetto, ma condividono una capacità.

◆ Implementare più interfacce

Una classe può implementare più interfacce.

```
interface Stampabile {  
    void stampa();  
}
```

```
interface Salvabile {  
    void salva();  
}
```

```
class Documento  
    implements Stampabile,  
               Salvabile {  
  
    ---  
  
    @Override  
    public void stampa() {  
    }  
  
    @Override  
    public void salva() {  
    }  
}
```

◆ Perché sono importanti

Java non permette l'ereditarietà multipla tra classi.

Questo codice non è valido:

```
class C extends A, B {  
}
```

Una classe può però implementare molte interfacce.

```
class C implements A, B, D {  
}
```

◆ Costanti nelle interfacce

Le variabili dichiarate in un'interfaccia sono automaticamente:

```
public static final
```

Esempio:

```
interface Costanti {  
    int MAX = 100;  
}
```


Utilizzo:

```
System.out.println(  
    Costanti.MAX  
);
```

◆ Default Methods

Dalla versione Java 8 è possibile definire metodi con implementazione.

```
interface Stampabile {  
    void stampa();  
  
    default void anteprima() {  
        System.out.println("Anteprima");  
    }  
}
```

Le classi possono utilizzare direttamente il metodo oppure ridefinirlo.

◆ Metodi Static

Un'interfaccia può contenere metodi static.

```
interface Utilita {  
    static void info() {  
        System.out.println("Info");  
    }  
}
```

Utilizzo:

```
Utilita.info();
```

◆ Interfacce funzionali

Un'interfaccia che contiene un solo metodo astratto prende il nome di:

```
Functional Interface
```

Esempio:

```
@FunctionalInterface
interface Operazione {

    int applica(int x);
}
```

Queste interfacce verranno utilizzate nel capitolo dedicato alle Lambda Expression.

◆ Buone pratiche

- usare nomi che descrivano una capacità;
- preferire le interfacce quando interessa il comportamento;
- utilizzare classi astratte quando esiste una relazione IS-A;
- mantenere le interfacce semplici e focalizzate.

✓ Riepilogo

- Un'interfaccia definisce un contratto.
- Si dichiara tramite `interface`.
- Una classe la implementa tramite `implements`.
- Le interfacce favoriscono il polimorfismo.
- Una classe può implementare più interfacce.
- Le interfacce supportano metodi `default` e `static`.
- Le interfacce funzionali sono alla base delle Lambda Expression.

Domande

Che cos'è un'interfaccia?

- A. Una classe speciale
- B. Un contratto che definisce comportamenti
- C. Un costruttore
- D. Un package

Quale parola chiave si usa per dichiarare un'interfaccia?

- A. abstract
- B. class
- C. interface
- D. contract

Quale parola chiave si usa per implementare un'interfaccia?

- A. extends
- B. implements
- C. inherit
- D. override

Un'interfaccia può essere istanziata?

- A. Sempre
- B. Solo se contiene metodi default
- C. Solo tramite costruttore
- D. No

Una classe può implementare più interfacce?

- A. No
- B. Solo due
- C. Sì
- D. Solo se è astratta

Quale relazione descrive meglio un'interfaccia?

A. IS-A

B. PART-OF

C. HAS-A

D. CAPABLE-OF

Quali sono automaticamente gli attributi di un'interfaccia?

- A. private static
- B. protected final
- C. public static final
- D. public abstract

Da quale versione di Java esistono i metodi default?

- A. Java 5
- B. Java 6
- C. Java 7
- D. Java 8

Che cos'è una Functional Interface?

- A. Un'interfaccia senza metodi
- B. Un'interfaccia con un solo metodo astratto
- C. Un'interfaccia astratta
- D. Una classe astratta

Qual è il principale vantaggio delle interfacce?

- A. Consentono l'ereditarietà multipla dei comportamenti
- B. Eliminano il polimorfismo
- C. Sostituiscono le classi
- D. Eliminano i metodi astratti

Esercizi

Esercizio 1

Creare l'interfaccia:

```
Stampabile
```

con il metodo:

```
void stampa();
```

Esercizio 2

Creare le classi:

Documento
Foto

che implementano `Stampabile` .

Esercizio 3

Creare una classe che implementi contemporaneamente:

```
Stampabile  
Salvabile
```

Esercizio 4

Confrontare:

```
abstract class Forma
```

e

```
interface Disegnabile
```

indicando differenze e casi d'uso.

Esercizio 5

Progettare un sistema con almeno tre classi che implementano la stessa interfaccia.

◆ Enum in Java

Una **enum** (abbreviazione di *enumeration*) è un tipo speciale di classe che rappresenta un insieme fisso di valori costanti.

Le enum sono state introdotte in Java 5 per sostituire l'utilizzo di costanti numeriche o stringhe poco sicure.

◆ Perché usare una enum

Supponiamo di voler rappresentare i giorni della settimana.

Una soluzione poco robusta potrebbe essere:

```
String giorno = "LUNEDI";
```

oppure:

```
int giorno = 1;
```

Queste soluzioni possono causare errori:

```
giorno = "LUNEDI"; // errore di battitura  
giorno = 99;       // valore non valido
```

Con una enum:

```
Giorno giorno = Giorno.LUNEDI;
```

◆ Dichiarazione di una enum

Si utilizza la parola chiave:

```
enum
```

Esempio:

```
public enum Giorno {  
  
    LUNEDI,  
    MARTEDI,  
    MERCOLEDI,  
    GIOVEDI,  
    VENERDI,  
    SABATO,  
    DOMENICA  
  
}
```

◆ Utilizzo di una enum

```
Giorno oggi = Giorno.LUNEDI;  
  
System.out.println(oggi);
```

Output:

```
LUNEDI
```

◆ Enum e Switch

Le enum vengono utilizzate molto spesso negli switch.

```
switch (oggi) {  
    case LUNEDI:  
        System.out.println("Inizio settimana");  
        break;  
  
    case VENERDI:  
        System.out.println("Quasi weekend");  
        break;  
  
    default:  
        System.out.println("Giorno normale");  
}
```

◆ Le enum sono classi

Anche se la sintassi è particolare, una enum è una vera classe Java.

Può contenere:

- attributi;
- costruttori;
- metodi.

◆ Enum con attributi

```
public enum Priorita {  
  
    BASSA(1),  
    MEDIA(2),  
    ALTA(3);  
  
    private int livello;  
  
    Priorita(int livello) {  
        this.livello = livello;  
    }  
  
    public int getLivello() {  
        return livello;  
    }  
}
```


Utilizzo:

```
System.out.println(  
    Priorita.ALTA.getLivello()  
);
```

Output:

```
3
```

◆ Costruttori nelle enum

Le enum possono avere costruttori.

```
Priorita(int livello) {  
    this.livello = livello;  
}
```

Tuttavia il costruttore viene utilizzato solo internamente dalla enum.

Non possiamo scrivere:

```
new Priorita(); // ERRORE
```

◆ Metodo values()

Ogni enum possiede automaticamente il metodo:

```
values()
```

che restituisce tutti i valori.

```
for (Giorno g : Giorno.values()) {  
    System.out.println(g);  
}
```

Output:

```
LUNEDI  
MARTEDI  
MERCOLEDI  
...
```

◆ Metodo `valueOf()`

Permette di ottenere una costante a partire dal nome.

```
Giorno g =  
    Giorno.valueOf("LUNEDI");
```

Output:

```
LUNEDI
```

Se il nome non esiste viene generata un'eccezione.

◆ Enum e confronto

Le enum possono essere confrontate tramite:

```
==
```

```
if (oggi == Giorno.LUNEDI) {  
    System.out.println("Inizio settimana");  
}
```

Questo è possibile perché ogni costante enum esiste in un'unica istanza.

◆ Esempi comuni di enum

```
Giorno  
Mese  
Stagione  
Priorita  
StatoOrdine  
RuoloUtente  
LivelloAccesso
```

Le enum sono particolarmente utili quando i valori possibili sono limitati e ben definiti.

◆ Enum e Classi

Possiamo vedere una enum come una classe speciale.

Classe

|
├── molti oggetti possibili

Enum

|
└── insieme fisso di oggetti

◆ Quando usare una enum

Utilizzare una enum quando:

- il numero dei valori è limitato;
- i valori sono noti in anticipo;
- non devono essere creati nuovi valori durante l'esecuzione;
- si vogliono evitare errori dovuti a stringhe o numeri.

◆ Buone pratiche

- utilizzare nomi significativi;
- usare le enum al posto di costanti numeriche;
- sfruttare gli switch con enum;
- aggiungere metodi solo quando realmente necessari.

✓ Riepilogo

- Una enum rappresenta un insieme fisso di costanti.
- Si dichiara tramite la parola chiave `enum`.
- Le enum sono vere classi Java.
- Possono contenere attributi, costruttori e metodi.
- Forniscono automaticamente `values()` e `valueOf()`.
- Rendono il codice più sicuro e leggibile.

Domande

Che cos'è una enum?

- A. Una sottoclasse speciale
- B. Un insieme fisso di costanti
- C. Un'interfaccia
- D. Un package

Quale parola chiave si usa per dichiarare una enum?

- A. constant
- B. enumeration
- C. enum
- D. final

Quale istruzione è corretta?

- A. `Giorno g = "LUNEDI";`
- B. `Giorno g = Giorno.LUNEDI;`
- C. `Giorno g = new Giorno();`
- D. `enum g = LUNEDI;`

Una enum può contenere metodi?

- A. No
- B. Solo static
- C. Solo default
- D. Sì

Quale metodo restituisce tutti i valori di una enum?

- A. getAll()
- B. values()
- C. valueOf()
- D. enumValues()

Quale metodo converte una stringa in una costante enum?

- A. parse()
- B. convert()
- C. valueOf()
- D. values()

È possibile creare una enum con `new` ?

- A. Sempre
- B. Solo se ha un costruttore pubblico
- C. No
- D. Solo nelle sottoclassi

Le enum possono essere utilizzate negli switch?

- A. No
- B. Solo da Java 17
- C. Solo se implementano un'interfaccia
- D. Sì

Quale tra questi è un buon candidato per una enum?

A. Persona

B. Automobile

C. StatoOrdine

D. Studente

Quale vantaggio offrono le enum?

- A. Eliminano il polimorfismo
- B. Rendono il codice più sicuro e leggibile
- C. Sostituiscono le classi
- D. Eliminano i costruttori

Esercizi

Esercizio 1

Creare una enum `Mese` contenente i dodici mesi dell'anno.

Esercizio 2

Creare una enum `Stagione` e utilizzarla in uno switch.

Esercizio 3

Creare una enum `Priorita` con livelli:

```
BASSA  
MEDIA  
ALTA
```

associando un valore numerico a ciascuna costante.

Esercizio 4

Stampare tutti i valori di una enum utilizzando `values()` .

Esercizio 5

Spiegare con parole proprie la differenza tra:

```
String stato = "ATTIVO";
```

e

```
Stato stato = Stato.ATTIVO;
```

◆ Record in Java

I **Record** sono stati introdotti in Java per semplificare la creazione di classi che hanno il solo scopo di contenere dati.

Un record è una classe speciale che genera automaticamente gran parte del codice normalmente necessario per una classe tradizionale.

◆ Il problema delle classi contenitore

Consideriamo una semplice classe `Persona` .

```
public class Persona {  
  
    private final String nome;  
    private final int eta;  
  
    public Persona(String nome, int eta) {  
        this.nome = nome;  
        this.eta = eta;  
    }  
  
    public String getNome() {  
        return nome;  
    }  
    ---  
    public int getEta() {  
        return eta;  
    }  
}
```

◆ Cos'è un Record

Un record permette di scrivere la stessa classe in forma compatta.

```
public record Persona(  
    String nome,  
    int eta  
) {  
}
```

In una sola riga vengono definiti:

- attributi;
- costruttore;
- getter;
- `equals()`;
- `hashCode()`;
- `toString()`.

◆ Utilizzo di un Record

```
Persona p =  
    new Persona("Mario", 30);  
  
System.out.println(p.nome());  
System.out.println(p.eta());
```

Output:

```
Mario  
30
```

◆ **Metodi generati automaticamente**

Per ogni componente del record Java genera automaticamente:

```
nome()  
eta()
```

Attenzione:

```
p.nome()
```

e non:

```
p.getNome()
```


◆ Metodo toString()

```
Persona p =  
    new Persona("Mario", 30);  
  
System.out.println(p);
```

Output:

```
Persona[nome=Mario, eta=30]
```

Il metodo viene generato automaticamente.

◆ Metodo equals()

```
Persona p1 =  
    new Persona("Mario", 30);  
  
Persona p2 =  
    new Persona("Mario", 30);  
  
System.out.println(  
    p1.equals(p2)  
);
```

Output:

```
true
```

L'uguaglianza viene calcolata in base ai valori contenuti.

◆ Immutabilità

I record sono immutabili.

```
public record Persona(  
    String nome,  
    int eta  
) {  
}
```

Dopo la creazione:

```
Persona p =  
    new Persona("Mario", 30);
```

non è possibile modificare i valori.

```
p.nome = "Luca"; // ERRORE
```

◆ Aggiungere metodi

Un record può contenere metodi personalizzati.

```
public record Persona(  
    String nome,  
    int eta  
) {  
  
    public String saluta() {  
        return "Ciao, sono " + nome;  
    }  
}
```

Utilizzo:

```
System.out.println(  
    p.saluta()  
);
```

Output:

```
Ciao, sono Mario
```

◆ Costruttore compatto

Possiamo aggiungere controlli sui dati.

```
public record Persona(  
    String nome,  
    int eta  
) {  
  
    public Persona {  
  
        if (eta < 0) {  
            throw new IllegalArgumentException(  
                "Età non valida"  
            );  
        }  
    }  
}
```

Questo costruttore prende il nome di:

Compact Constructor

◆ Record e DTO

Uno degli utilizzi più comuni è la realizzazione di:

DTO
Data Transfer Object

ovvero oggetti utilizzati per trasferire dati tra:

- livelli dell'applicazione;
- API REST;
- database;
- servizi esterni.

◆ Limitazioni dei Record

I record hanno alcune restrizioni.

Non possono essere estesi

```
record Persona(  
    String nome  
) {  
}
```

è implicitamente:

```
final
```

Non possono avere setter

Essendo immutabili:

```
setNome(...)
```

non avrebbe senso.

Non possono aggiungere nuovi campi di istanza

I dati principali devono essere dichiarati nell'intestazione del record.

◆ Record vs Classe Tradizionale

Classe Tradizionale	Record
Molto codice	Poco codice
Getter manuali	Automatici
equals manuale	Automatico
hashCode manuale	Automatico
toString manuale	Automatico
Può essere mutabile	Immutabile

◆ Quando usare un Record

I record sono ideali quando:

- l'oggetto contiene principalmente dati;
- i dati non devono cambiare;
- si vuole ridurre il codice ripetitivo;
- si desidera un DTO compatto e leggibile.

◆ Quando NON usare un Record

È preferibile una classe tradizionale quando:

- l'oggetto possiede stato modificabile;
- servono setter;
- è necessaria una gerarchia di ereditarietà;
- la logica applicativa è predominante rispetto ai dati.

◆ Buone pratiche

- usare i record per rappresentare dati;
- mantenerli semplici;
- sfruttare l'immutabilità;
- evitare di trasformarli in classi complesse.

✓ Riepilogo

- Un record è una classe speciale introdotta nelle versioni moderne di Java.
- Riduce drasticamente il codice necessario per rappresentare dati.
- Genera automaticamente costruttore, getter, `equals`, `hashCode` e `toString`.
- È immutabile.
- È particolarmente adatto ai DTO.
- Non supporta l'ereditarietà tradizionale.

Domande

Che cos'è un Record?

- A. Una classe astratta
- B. Una classe speciale per rappresentare dati
- C. Un'interfaccia
- D. Una enum

In quale versione i Record sono diventati definitivi?

- A. Java 8
- B. Java 11
- C. Java 16
- D. Java 21

Quale metodo viene generato automaticamente?

- A. destroy()
- B. execute()
- C. equals()
- D. cloneAll()

Come si accede al campo `nome` di un record?

- A. `getNome()`
- B. `nome()`
- C. `Nome()`
- D. `valueOf()`

I Record sono:

A. Mutabili

B. Astratti

C. Immutabili

D. Statici

Un Record può essere esteso?

- A. Sempre
- B. Solo con extends
- C. Solo se abstract
- D. No

Che cos'è un Compact Constructor?

- A. Un costruttore generato automaticamente
- B. Un costruttore utilizzato per validazioni
- C. Un metodo static
- D. Un setter speciale

Quale utilizzo è tipico dei Record?

- A. DTO
- B. Thread
- C. Driver JDBC
- D. Package

Un Record può contenere metodi?

- A. No
- B. Solo static
- C. Solo private
- D. Sì

Quale vantaggio principale offrono i Record?

- A. Maggiore ereditarietà
- B. Meno codice boilerplate
- C. Più performance della JVM
- D. Eliminano il polimorfismo

Esercizi

Esercizio 1

Creare un record:

```
record Punto(int x, int y)
```

e stampare due istanze.

Esercizio 2

Creare un record:

```
record Prodotto(  
    String nome,  
    double prezzo  
)
```

e visualizzarne i dati.

Esercizio 3

Aggiungere un metodo:

```
double prezzoConIVA()
```

al record `Prodotto` .

Esercizio 4

Creare un compact constructor che impedisca prezzi negativi.

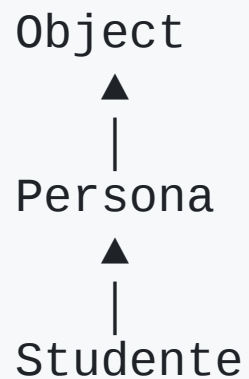
Esercizio 5

Convertire una classe DTO tradizionale in un record.

◆ La classe Object in Java

La classe `Object` è la superclasse di tutte le classi Java.

Questo significa che ogni classe definita in Java eredita direttamente o indirettamente da `Object`.



Anche se non scriviamo esplicitamente:

```
class Persona extends Object {  
}
```

l'ereditarietà da `Object` avviene automaticamente.

◆ Perché è importante

Poiché tutte le classi derivano da `Object`, ogni oggetto Java possiede automaticamente alcuni metodi fondamentali.

Tra i più importanti:

```
toString()  
equals()  
hashCode()  
getClass()
```

◆ **Metodo toString()**

Restituisce una rappresentazione testuale dell'oggetto.

Comportamento predefinito

```
class Persona {  
    String nome;  
  
    Persona(String nome) {  
        this.nome = nome;  
    }  
}
```

```
Persona p = new Persona("Mario");  
---  
  
System.out.println(p);
```

Output:

```
Persona@15db9742
```

Il risultato non è molto utile.

◆ Override di toString()

Normalmente il metodo viene ridefinito.

```
class Persona {  
    private String nome;  
  
    public Persona(String nome) {  
        this.nome = nome;  
    }  
  
    @Override  
    public String toString() {  
        return "Persona[nome=" + nome + "]";  
    }  
}
```


Output:

```
Persona[nome=Mario]
```

◆ **Metodo equals()**

Serve a confrontare due oggetti.

Confronto tramite ==

```
Persona p1 =  
    new Persona("Mario");  
  
Persona p2 =  
    new Persona("Mario");  
  
System.out.println(p1 == p2);
```

Output:

```
false
```

Perché i due riferimenti puntano a oggetti diversi.

◆ Override di equals()

```
@Override
public boolean equals(Object obj) {

    if (this == obj)
        return true;

    if (!(obj instanceof Persona))
        return false;

    Persona p = (Persona) obj;

    return nome.equals(p.nome);
}
```

Utilizzo:

```
System.out.println(  
    p1.equals(p2)  
);
```

Output:

```
true
```

◆ Metodo hashCode()

Restituisce un numero intero associato all'oggetto.

```
int codice =  
    p1.hashCode();
```

È molto utilizzato da strutture dati come:

```
HashMap  
HashSet  
Hashtable
```

◆ Regola fondamentale

Se si ridefinisce:

```
equals()
```

si dovrebbe ridefinire anche:

```
hashCode()
```

per mantenere la coerenza.

Esempio:

```
@Override  
public int hashCode() {  
    return Objects.hash(nome);  
}
```

◆ Metodo getClass()

Permette di conoscere la classe dell'oggetto durante l'esecuzione.

```
Persona p =  
    new Persona("Mario");  
  
System.out.println(  
    p.getClass()  
);
```

Output:

```
class Persona
```

◆ Informazioni sulla classe

```
Class<?> c =  
    p.getClass();  
  
System.out.println(  
    c.getSimpleName()  
);
```

Output:

```
Persona
```

◆ Object e Polimorfismo

Poiché tutte le classi derivano da `Object`, possiamo scrivere:

```
Object obj =  
    new Persona("Mario");
```

oppure:

```
Object obj =  
    new String("ciao");
```

oppure:

```
Object obj =  
    Integer.valueOf(10);
```

Tutti sono oggetti Java validi.

◆ Record e Object

Nel capitolo precedente abbiamo visto i Record.

```
record Persona(  
    String nome,  
    int eta  
) {  
}
```

Il compilatore genera automaticamente:

```
toString()  
equals()  
hashCode()
```

proprio sfruttando i meccanismi ereditati da `Object` .

◆ Altri metodi della classe Object

La classe `Object` contiene anche altri metodi meno utilizzati.

clone()

Permette di creare copie di oggetti.

wait()

Sospende un thread.

notify()

Risveglia un thread in attesa.

notifyAll()

Risveglia tutti i thread in attesa.

finalize()

Storicamente utilizzato per la pulizia delle risorse.

Oggi è deprecato e non dovrebbe essere utilizzato.

◆ Buone pratiche

- ridefinire `toString()` per facilitare il debug;
- ridefinire `equals()` quando il confronto deve basarsi sui dati;
- ridefinire `hashCode()` insieme a `equals()` ;
- utilizzare `getClass()` per ottenere informazioni a runtime;
- evitare l'uso di `finalize()` .

✓ Riepilogo

- `Object` è la superclasse di tutte le classi Java.
- Ogni oggetto eredita automaticamente i metodi di `Object`.
- `toString()` produce una rappresentazione testuale.
- `equals()` confronta gli oggetti.
- `hashCode()` genera un codice hash.
- `getClass()` restituisce informazioni sulla classe.
- I Record generano automaticamente diversi metodi ereditati da `Object`.

Domande

Che cos'è la classe `Object` ?

- A. Una interfaccia
- B. Una classe astratta
- C. La superclasse di tutte le classi Java
- D. Un package

Quale metodo produce una rappresentazione testuale di un oggetto?

- A. print()
- B. show()
- C. display()
- D. toString()

Che cosa confronta l'operatore `==` applicato a oggetti?

- A. Il contenuto degli oggetti
- B. I riferimenti agli oggetti
- C. I valori degli attributi
- D. Gli hashCode

Quale metodo viene utilizzato per confrontare il contenuto di due oggetti?

- A. compare()
- B. equals()
- C. hashCode()
- D. getClass()

Quale metodo dovrebbe essere ridefinito insieme a `equals()` ?

- A. `clone()`
- B. `finalize()`
- C. `hashCode()`
- D. `wait()`

Quale metodo restituisce informazioni sulla classe di un oggetto?

- A. classOf()
- B. getType()
- C. getClass()
- D. className()

Quale metodo è stato deprecato nelle versioni moderne di Java?

- A. equals()
- B. toString()
- C. hashCode()
- D. finalize()

Quale struttura utilizza frequentemente `hashCode()` ?

- A. Array
- B. Scanner
- C. HashMap
- D. StringBuilder

Quale affermazione è corretta?

- A. Solo le classi astratte ereditano da Object
- B. Solo i Record ereditano da Object
- C. Tutte le classi Java ereditano da Object
- D. Le interfacce ereditano da Object

Quale metodo viene generato automaticamente nei Record?

- A. destroy()
- B. execute()
- C. equals()
- D. notify()

Esercizi

Esercizio 1

Creare una classe `Persona` e ridefinire `toString()`.

Esercizio 2

Ridefinire `equals()` confrontando nome e cognome.

Esercizio 3

Aggiungere una corretta implementazione di `hashCode( )` .

Esercizio 4

Utilizzare `getClass()` per stampare il nome della classe di un oggetto.

Esercizio 5

Spiegare la differenza tra:

```
p1 == p2
```

e

```
p1.equals(p2)
```

◆ Eccezioni in Java

Le **eccezioni** (*Exceptions*) sono eventi anomali che interrompono il normale flusso di esecuzione di un programma.

Quando si verifica un errore durante l'esecuzione, Java genera un oggetto che descrive il problema.

Questo oggetto prende il nome di:

Eccezione

◆ Perché servono le eccezioni

Senza la gestione delle eccezioni un programma potrebbe terminare improvvisamente.

Esempio:

```
int a = 10;  
int b = 0;  
  
System.out.println(a / b);
```

Output:

```
Exception in thread "main"  
java.lang.ArithmeticException
```

Il programma termina immediatamente.

◆ Gestire un'eccezione

Per intercettare un errore si utilizza un blocco:

```
try {  
}  
catch (...) {  
}
```

Esempio:

```
try {  
    ---  
  
    int risultato = 10 / 0;  
} catch (ArithmeticException e) {  
    System.out.println(  
        "Divisione per zero"  
    );  
}
```

Output:

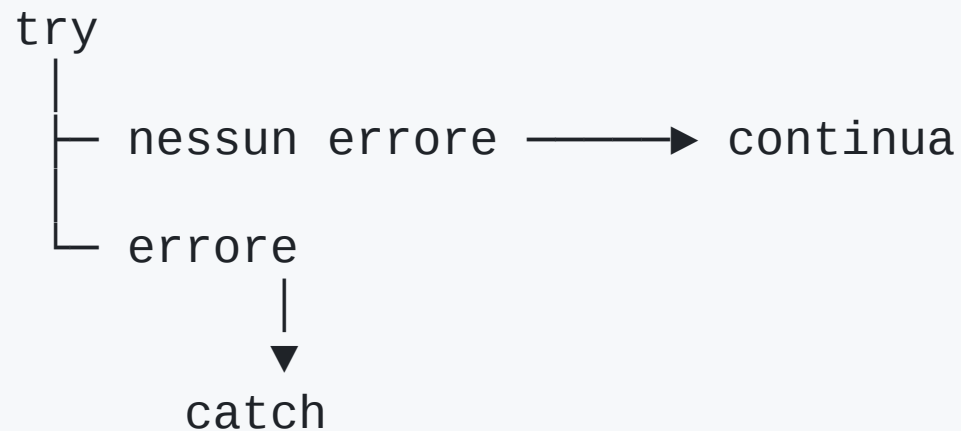
```
Divisione per zero
```

Il programma continua a funzionare.

◆ Struttura di try-catch

```
try {  
    // codice rischioso  
} catch (TipoEccezione e) {  
    // gestione errore  
}
```

Schema:



◆ **Eccezioni comuni**

ArithmeticException

```
int x = 10 / 0;
```

NullPointerException

```
String nome = null;  
nome.length();
```

ArrayIndexOutOfBoundsException

```
int[] numeri = {1,2,3};  
  
System.out.println(  
    numeri[10]  
);
```

NumberFormatException

```
Integer.parseInt("abc");
```

◆ Più blocchi catch

Possiamo gestire errori differenti.

```
try {  
    // codice  
} catch (ArithmeticException e) {  
    System.out.println(  
        "Errore matematico"  
    );  
} catch (NumberFormatException e) {  
    System.out.println(  
        "Numero non valido"  
    );  
}
```

◆ Il blocco finally

Il blocco:

```
finally
```

viene eseguito sempre.

```
try {  
    System.out.println("Try");  
} finally {  
    System.out.println("Finally");  
}
```

Output:

```
Try  
Finally
```

◆ Utilità di finally

È spesso utilizzato per:

- chiudere file;
- chiudere connessioni;
- liberare risorse.

```
try {  
    // utilizzo risorsa  
} finally {  
    // chiusura risorsa  
}
```


◆ La gerarchia delle eccezioni

Tutte le eccezioni derivano da:

Throwable

Schema semplificato:

```
Throwable
|
├── Error
└── Exception
    └── RuntimeException
```

◆ **Eccezioni controllate e non controllate**

Checked Exception

Devono essere gestite obbligatoriamente.

Esempio:

`IOException`

Unchecked Exception

Derivano da:

`RuntimeException`

e non devono essere necessariamente gestite.

Esempi:

```
NullPointerException  
ArithmeticException  
NumberFormatException
```

◆ La parola chiave throw

Permette di generare manualmente un'eccezione.

```
if (eta < 0) {  
    throw new IllegalArgumentException(  
        "Età non valida"  
    );  
}
```

◆ La parola chiave throws

Indica che un metodo può generare un'eccezione.

```
public void leggiFile()  
    throws IOException {  
  
}
```

Chi utilizza il metodo dovrà gestire l'errore.

◆ Eccezioni personalizzate

Possiamo creare nuove eccezioni.

```
public class EtaNonValidaException
    extends Exception {

    public EtaNonValidaException(
        String messaggio) {

        super(messaggio);
    }
}
```

Utilizzo:

```
throw new EtaNonValidaException(  
    "Età negativa"  
);
```

◆ Try-With-Resources

Nelle versioni moderne di Java si utilizza spesso:

```
try (  
    Scanner sc =  
        new Scanner(file)  
) {  
  
}
```

Le risorse vengono chiuse automaticamente.

◆ Buone pratiche

- gestire solo le eccezioni che si conoscono;
- non ignorare i blocchi catch;
- utilizzare messaggi chiari;
- usare eccezioni personalizzate quando necessario;
- preferire il try-with-resources per file e stream.

✓ Riepilogo

- Le eccezioni rappresentano errori durante l'esecuzione.
- Si gestiscono tramite `try` e `catch`.
- Il blocco `finally` viene sempre eseguito.
- `throw` genera un'eccezione.
- `throws` dichiara un'eccezione.
- Le eccezioni possono essere checked o unchecked.
- È possibile creare eccezioni personalizzate.

Domande

Che cos'è un'eccezione?

- A. Un package Java
- B. Un evento anomalo durante l'esecuzione
- C. Un tipo di variabile
- D. Un costruttore speciale

Quale blocco contiene il codice che potrebbe generare errori?

- A. catch
- B. finally
- C. try
- D. throw

Quale blocco gestisce l'eccezione?

- A. catch
- B. try
- C. throws
- D. import

Quale blocco viene eseguito sempre?

- A. catch
- B. finally
- C. throw
- D. if

Quale eccezione viene generata da una divisione per zero?

- A. NullPointerException
- B. IOException
- C. ArithmeticException
- D. NumberFormatException

Quale eccezione viene generata accedendo a un riferimento null?

- A. ArithmeticException
- B. NullPointerException
- C. IOException
- D. ClassCastException

Che cosa fa la parola chiave `throw` ?

- A. Gestisce un'eccezione
- B. Genera un'eccezione
- C. Ignora un'eccezione
- D. Chiude un file

Che cosa indica la parola chiave `throws` ?

- A. Che il metodo può generare un'eccezione
- B. Che il metodo cattura un'eccezione
- C. Che il metodo è static
- D. Che il metodo è astratto

Quale classe è la radice della gerarchia delle eccezioni?

- A. Exception
- B. RuntimeException
- C. Throwable
- D. Error

Quale eccezione è unchecked?

- A. IOException
- B. SQLException
- C. FileNotFoundException
- D. NullPointerException

Esercizi

Esercizio 1

Gestire una divisione per zero tramite `try-catch` .

Esercizio 2

Gestire una `NumberFormatException` durante la conversione di una stringa.

Esercizio 3

Creare un metodo che lanci una `IllegalArgumentException` se l'età è negativa.

Esercizio 4

Creare una eccezione personalizzata `SaldoNegativoException` .

Esercizio 5

Realizzare un programma che utilizzi:

```
try  
catch  
finally
```

e stampi messaggi differenti nei tre blocchi.

◆ Nested Classes in Java

In Java è possibile definire una classe all'interno di un'altra classe.

Queste classi prendono il nome di:

```
Nested Classes  
Classi Annidate  
Classi Innestate
```

Le nested classes permettono di organizzare meglio il codice quando una classe è strettamente legata a un'altra.

◆ Perché usare le Nested Classes

Supponiamo di avere una classe:

```
class Automobile {  
}
```

e una classe utilizzata esclusivamente da essa:

```
class Motore {  
}
```

Se `Motore` non ha significato al di fuori di `Automobile`, può essere opportuno dichiararla all'interno della classe principale.

```
class Automobile {  
    class Motore {  
    }  
}
```


◆ Tipologie di Nested Classes

Java distingue due categorie principali:

Nested Classes

|

├ Inner Classes

└ Static Nested Classes

◆ Inner Classes

Una **Inner Class** è una classe dichiarata all'interno di un'altra classe.

Ha accesso diretto a tutti i membri della classe esterna, compresi quelli privati.

◆ Esempio

```
class Automobile {  
    private String marca = "Fiat";  
    class Motore {  
        public void mostraMarca() {  
            System.out.println(marca);  
        }  
    }  
}
```

La classe interna può accedere direttamente a:

marca

anche se è privata.

◆ Creazione di una Inner Class

Per creare una inner class occorre prima creare l'oggetto esterno.

```
Automobile auto =  
    new Automobile();  
  
Automobile.Motore motore =  
    auto.new Motore();
```

◆ Schema

```
classDiagram
    class Automobile {
        class Motore
    }
```

Automobile
└─ Motore

La classe interna dipende da una specifica istanza della classe esterna.

◆ Quando usare una Inner Class

È utile quando:

- la classe viene utilizzata esclusivamente dalla classe esterna;
- serve accedere facilmente ai dati privati;
- si vuole migliorare l'incapsulamento.

◆ Static Nested Classes

Una **Static Nested Class** è una classe annidata dichiarata con:

```
static
```

Non dipende da un'istanza della classe esterna.

◆ Esempio

```
class Automobile {  
    static class Costanti {  
        public static final int RUOTE = 4;  
    }  
}
```

◆ Creazione

Non serve creare un oggetto della classe esterna.

```
Automobile.Costanti c =  
    new Automobile.Costanti();
```

oppure:

```
System.out.println(  
    Automobile.Costanti.RUOTE  
);
```

◆ Accesso ai membri

Una static nested class può accedere direttamente solo ai membri statici della classe esterna.

```
class Automobile {  
    static String categoria =  
        "Veicolo";  
    static class Info {  
        void stampa() {  
            System.out.println(  
                categoria  
            );  
        }  
    }  
}
```

◆ Inner Class vs Static Nested Class

Inner Class	Static Nested Class
Dipende da un oggetto esterno	Non dipende da un oggetto
Accede ai membri privati	Accede direttamente solo ai membri statici
Richiede un'istanza della classe esterna	Non richiede un'istanza
Più legata all'oggetto	Più simile a una normale classe

◆ Esempio completo

```
class Università {  
  
    private String nome =  
        "Politecnico";  
  
    class Studente {  
  
        public void mostra() {  
  
            System.out.println(  
                nome  
            );  
        }  
    }  
    ---  
  
    static class Costanti {  
  
        static final int MAX_STUDENTI =  
            1000;  
    }  
}
```

◆ **Vantaggi delle Nested Classes**

- migliore organizzazione del codice;
- maggiore incapsulamento;
- riduzione della visibilità non necessaria;
- raggruppamento logico di classi correlate.

◆ Quando NON usarle

Evitare le nested classes quando:

- la classe può essere riutilizzata altrove;
- la dipendenza dalla classe esterna non è reale;
- la struttura rende il codice più difficile da leggere.

Riepilogo

- Una nested class è una classe definita all'interno di un'altra classe.
- Le principali tipologie sono:
 - Inner Class
 - Static Nested Class
- Le inner class accedono ai membri della classe esterna.
- Le static nested class sono simili a normali classi statiche.
- Le nested classes migliorano organizzazione e incapsulamento.

Domande

Che cos'è una Nested Class?

- A. Una classe astratta
- B. Una classe definita all'interno di un'altra classe
- C. Una sottoclasse
- D. Una interfaccia

Quale tipologia di nested class dipende da un'istanza della classe esterna?

- A. Static Nested Class
- B. Enum
- C. Record
- D. Inner Class

Una Inner Class può accedere ai membri privati della classe esterna?

- A. No
- B. Solo se protected
- C. Solo se static
- D. Sì

Come si dichiara una Static Nested Class?

- A. final class
- B. abstract class
- C. static class
- D. private class

Per creare una Inner Class è necessario:

- A. usare extends
- B. creare prima un oggetto della classe esterna
- C. usare implements
- D. usare super

Una Static Nested Class può essere istanziata senza creare l'oggetto esterno?

- A. No
- B. Solo con reflection
- C. Sì
- D. Solo se public

Quale vantaggio offrono le Nested Classes?

- A. Aumentano la velocità della JVM
- B. Migliorano l'organizzazione del codice
- C. Eliminano il polimorfismo
- D. Sostituiscono le interfacce

Quale delle seguenti è una Inner Class?

A.

```
class A {  
    class B {  
    }  
}
```


B.

```
interface B {  
}
```

C.

```
record B() {  
}
```

D.

```
enum B {  
}
```

Quale delle seguenti è una Static Nested Class?

A.

```
class A {  
    static class B {  
    }  
}
```

B.

```
class B {  
}
```

C.

```
interface B {  
}
```

D.

```
record B() {  
}
```

Quando è opportuno usare una Nested Class?

- A. Quando la classe è strettamente legata alla classe esterna
- B. Sempre
- C. Solo nelle interfacce
- D. Solo nelle classi astratte

Esercizi

Esercizio 1

Creare una classe `Automobile` contenente una inner class `Motore` .

Esercizio 2

Creare una classe `Università` con una inner class `Studente` .

Esercizio 3

Creare una static nested class `Costanti` contenente alcune costanti pubbliche.

Esercizio 4

Spiegare la differenza tra:

```
class Inner {  
}
```

e

```
static class Inner {  
}
```

Esercizio 5

Realizzare una classe esterna con:

- una inner class;
- una static nested class;

e mostrare come istanziarle entrambe.